

Algoritmi e Strutture Dati - Prova d'esame

20/12/06

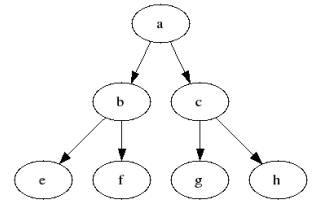
Esercizio 1 - Lunghezza di cammino

La lunghezza di cammino in un albero radicato (detto anche albero con radice) è la somma delle distanze dei nodi dalla radice. Ad esempio, la lunghezza del cammino dell'albero a fianco è pari a 10:

Scrivere un algoritmo in pseudo-codice (con commenti) per calcolare la lunghezza di cammino in un albero radicato T .

Discutere la complessità in tempo dell'algoritmo proposto in funzione del numero di nodi dell'albero.

Soluzione



1. Supponiamo che l'albero di ingresso sia rappresentato da una famiglia di nodi V e, per ogni $v \in V$, dalla lista $L(v)$ dei figli di v . Denotiamo inoltre con r la radice dell'albero. L'algoritmo esegue una semplice visita in profondità dell'albero, definita mediante una procedura ricorsiva che chiamiamo **Calcola**; quest'ultima su input $v \in V$ richiama se stessa su tutti i figli di v e aggiorna due variabili globali: $SumDist$ rappresenta la somma delle distanze dei nodi già visitati, mentre $CurDist$ rappresenta la distanza dalla radice del nodo corrente v .

Formalmente l'algoritmo è descritto dalle seguenti procedure:

Algorithm 1 LUNGHEZZA_CAMMINO(R)

```
1:  $SumDist \leftarrow 0$ 
2:  $CurDist \leftarrow 0$ 
3: Calcola( $r$ )
4: return  $SumDist$ 
```

Algorithm 2 CALCOLA(v)

```
1:  $SumDist \leftarrow SumDist + CurDist$ 
2:  $CurDist \leftarrow CurDist + 1$ 
3: for all  $w \in L(v)$  do
4:   Calcola( $w$ )
5:  $CurDist \leftarrow CurDist - 1$ 
```

2. Denotando con n il numero di nodi dell'albero, il tempo di calcolo è $\Theta(n)$ poiché tutti i nodi vengono visitati una volta.

Esercizio 2 - Ricorrenza

Trovare limiti asintotici superiore e inferiore più stretti possibile per la seguente relazione di ricorrenza, utilizzando il metodo di sostituzione.

$$T(n) = T(n - 2) + 2 \log n$$

Soluzione

La nostra guess è $T(n) = \Theta(n \log n)$.

Dimostriamo prima limite superiore e poi limite inferiore.

Limite superiore: $T(n) = O(n \log n)$.

Utilizziamo il metodo di sostituzione. La nostra ipotesi induttiva è che $T(m) \leq cm \log m$ per tutti gli $m < n$ e per qualche costante $c > 0$, e vogliamo dimostrare che la proprietà è vera per n .

$$\begin{aligned} T(n) &= T(n - 2) + 2 \log n \\ &\leq c(n - 2) \log(n - 2) + 2 \log n \\ &\leq c(n - 2) \log n + \log n \\ &= (cn - 2c + 2) \log n \\ &= cn \log n + (2 - 2c) \log n \end{aligned}$$

Siccome vogliamo che $cn \log n + (2 - 2c) \log n \leq nc \log n$, ciò è vero per $c \geq 1$.
Quindi $T(n) = O(n \log n)$ e abbiamo trovato un limite superiore.

Limite inferiore: $T(n) = \Omega(n \log n)$.

Utilizziamo il metodo di sostituzione. La nostra ipotesi induttiva è che $T(m) \geq cm \log m + dm$ per tutti gli $m < n$ e per qualche costante $c, d > 0$, e vogliamo dimostrare che la proprietà è vera per n . Assumiamo che $n \geq 4$, in modo da avere le seguenti disuguaglianze che useremo nel seguito:

1. $\log(n - 2) \geq \log(n/2)$
2. $n/2 \geq \log n$
3. $n/2 \geq 2$

$$\begin{aligned}
 T(n) &= T(n - 2) + 2 \log n \\
 &\geq c(n - 2) \log(n - 2) + d(n - 2) + 2 \log n \\
 &= cn \log(n - 2) - 2c \log(n - 2) + dn - 2d + 2 \log n \\
 &\geq cn \log(n - 2) - 2c \log(n) + dn - 2d + 2 \log n \\
 &\quad (\text{siccome } -\log n \leq -\log(n - 2)) \\
 &= cn \log(n - 2) - 2(c - 1) \log(n) + dn - 2d \\
 &\geq cn \log(n/2) - 2(c - 1) \log(n) + dn - 2d \\
 &\quad (\text{peruguaglianza (1) introdotta sopra}) \\
 &= cn \log(n) - cn \log(2) - 2(c - 1) \log(n) + dn - 2d \\
 &= cn \log(n) - cn - 2(c - 1) \log(n) + dn - 2d
 \end{aligned}$$

Siccome vogliamo che $cn \log(n) - cn - 2(c - 1) \log(n) + dn - 2d \geq cn \log n$, ciò è vero se $-cn - 2(c - 1) \log(n) + dn - 2d \geq 0$ o, in maniera equivalente, $dn \geq cn + 2(c - 1) \log(n) + 2d$.

Possiamo scegliere c e d e due valori per cui questa disuguaglianza è vera sono $c = 1$ e $d = 3$ (per $n \geq 3$).

Quindi $T(n) = \Omega(n \log n)$ e abbiamo trovato un limite inferiore.

Siccome $T(n) = O(n \log n)$ e $T(n) = \Omega(n \log n)$, $T(n) = \Theta(n \log n)$.

Esercizio 3 - Individuazione delle componenti fortemente connesse di un grafo

Sia dato un grafo orientato G costituito da 8 nodi etichettati con le lettere da a a h . Gli archi sono forniti come liste di adiacenze.

$a.\text{adj} = b, e$

$b.\text{adj} = c, f$

$c.\text{adj} = f, d, h, g$

$d.\text{adj} = h$

$e.\text{adj} = f, a$

$f.\text{adj} = b$

$g.\text{adj} = h$

$h.\text{adj} = d$

Applicare l'algoritmo visto a lezione per individuare le componenti fortemente connesse (strongly connected component) del grafo G . L'algoritmo consiste in una prima visita in profondità sul grafo G e in una seconda visita in profondità sul grafo trasposto G^T .

Nota bene: se il vostro numero di matricola è pari, iniziate la visita a partire dal nodo b ; se il vostro numero di matricola è dispari, iniziate la visita a partire dal nodo c . Nel caso di "ripartenza" (ovvero, se la visita non riesce a raggiungere tutti i nodi), continuate a partire dal nodo a .

Nota bene: nella visita in profondità, l'ordine in cui i nodi adiacenti ad un nodo vengono visitati è esattamente quello in cui compaiono nella lista di adiacenza; ad esempio, per quanto riguarda il nodo b verranno visitati prima il nodo adiacente c e poi il nodo adiacente f .

Ciò che va indicato in maniera chiara sono:

- i tempi di fine visita di ogni nodo (relativi alla prima visita in profondità)
- le componenti fortemente connesse individuate dalla seconda visita in profondità. Le componenti fortemente connesse devono essere descritte come alberi depth first.

È consigliato ma non richiesto disegnare il grafo G e le componenti fortemente connesse.

Soluzione

Il grafo di input con i valori di discovery e finish per le visite che partono da b e c .

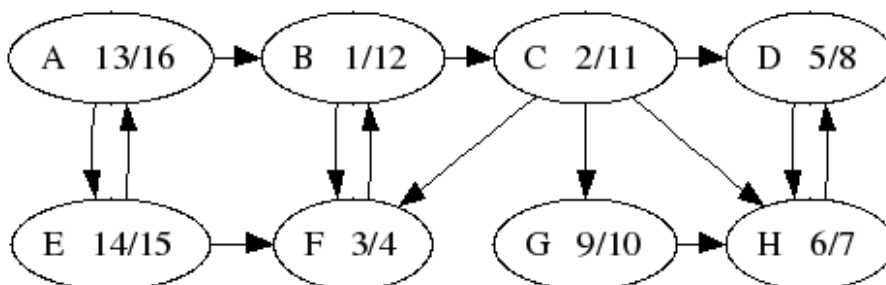


Figura 1: Grafo con tempi di discovery e finish per la visita che parte dal nodo b

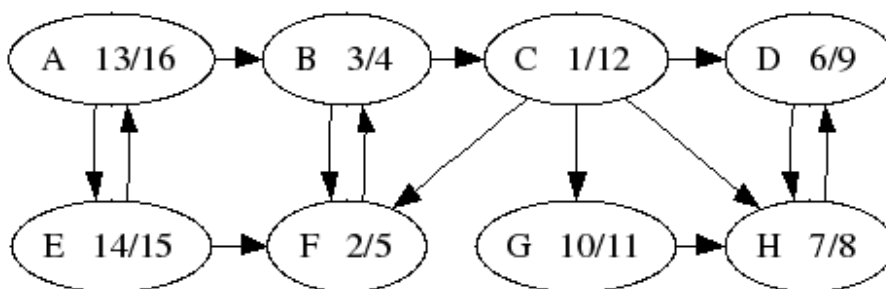


Figura 2: Grafo con tempi di discovery e finish per la visita che parte dal nodo c

Strongly connected components in ordine di individuazione, per la visita che parte dal nodo b . Il primo nodo dell'insieme e' la radice dell'albero depth-first.

$\{a, e\}$
 $\{b, f, c\}$
 $\{g\}$
 $\{d, h\}$

Strongly connected components in ordine di individuazione, per la visita che parte dal nodo c . Il primo nodo dell'insieme e' la radice dell'albero depth-first.

$\{a, e\}$
 $\{c, b, f\}$
 $\{g\}$
 $\{d, h\}$

Esercizio 4 - Massima somma di sottovettori

Progettare un algoritmo che, preso un vettore V di n interi (positivi o negativi), trovi un sottovettore (una sequenza di elementi **consecutivi** del vettore) la somma dei cui elementi sia massima. Cercare di ottenere l'algoritmo più efficiente possibile. Spiegare l'idea alla base dell'algoritmo e scrivere lo pseudocodice, commentato.

Suggerimento L'algoritmo banale, forza bruta, che analizza tutti i possibili casi ha complessità $O(n^3)$. È possibile scendere a $O(n^2)$ salvando informazione già calcolata al fine di evitare di ricalcolarla. È possibile fare di meglio, $O(n \log n)$, con una strategia divide and conquer. È infine possibile progettare un algoritmo che risolve in problema in tempo $O(n)$. Discutere la complessità dell'algoritmo proposto.

Soluzione

La soluzione che descrive i vari possibili algoritmi e le varie intuizioni per passare da $O(n^3)$ a $O(n)$ sono ben descritti (in inglese) nel documento PDF scaricabile a <http://sra.itc.it/people/massa/AlgorithmDesignTechniques.pdf>

La descrizione di questo problema e dei vari algoritmi che la risolvono è apparsa per la prima volta in "Programming Pearls" di Jon Bentley.

Algoritmi e Strutture Dati - Prova d'esame

08/01/07

Esercizio 1

Dimostrare che:

$$\sum_{i=1}^n \log i = \Theta(n \log n)$$

Soluzione, parte O Dalla definizione di Θ , dobbiamo provare che esistono due valori $c > 0$ e $n_0 > 0$ tali che:

$$\sum_{i=1}^n \log i \leq cn \log n, \forall n > n_0$$

Facile:

$$\begin{aligned} \sum_{i=1}^n \log i &\leq \sum_{i=1}^n \log n \\ &= n \log n \end{aligned}$$

Soluzione, parte Ω Dalla definizione di Θ , dobbiamo provare che esistono due valori $c > 0$ e $n_0 > 0$ tali che:

$$\sum_{i=1}^n \log i \geq cn \log n, \forall n > n_0$$

Lo proviamo in questo modo:

$$\begin{aligned} \sum_{i=1}^n \log i &= \log 1 + \dots + \log\left(\frac{n}{2} - 1\right) + \log \frac{n}{2} + \dots + \log n \\ &\geq 0 + \dots + 0 + \log \frac{n}{2} + \dots + \log \frac{n}{2} \\ &= \frac{n}{2} \log \frac{n}{2} \\ &= \frac{n}{2} \log n - n \log 2 \\ &= \frac{n}{2} \log n - n \\ &= \Omega(n \log n) \end{aligned}$$

Esercizio 2

Sia dato un vettore A di n valori numerici. Scrivere un algoritmo che *verifica* se tale vettore è un max-heap di n elementi oppure no. L'algoritmo effettua semplicemente la verifica, senza modificare lo heap.

Algorithm 1 CHECKMAXHEAP($A[1..n]$)

```
1: for  $i := 2$  to  $n$  do
2:   if  $A[i/2] < A[i]$  then
3:     return false
4: return true
```

Soluzione

Esercizio 3

Si consideri il problema seguente. Siano dati n sciatori di altezza $p_1, \dots, p_n$, e n paia di sci di lunghezza $s_1, \dots, s_n$. Il problema è assegnare ad ogni sciatore un paio di sci, in modo da minimizzare la differenza totale fra l'altezza dello sciatore e la lunghezza della persona; ovvero, se allo sciatore i è dato il paio di sci $h(i)$, minimizzare la seguente quantità:

$$\sum_{i=1}^n |p_i - s_{h(i)}|$$

Si consideri il seguente algoritmo greedy. Si individui uno sciatore e un paio di sci in modo tale che la differenza di altezza sia minimizzata. Si assegni allo sciatore questo paio di sci. Ripetere con gli sciatori e gli sci restanti fino a quando non si è terminato. Provare che questo algoritmo non è corretto (suggerimento: trovare un insieme di valori di altezza/lunghezza per cui l'algoritmo termina in maniera non corretta).

Soluzione Si consideri il seguente input: $p = \{5, 10\}$, $s = \{9, 14\}$. Secondo l'algoritmo, associamo $p[2]/s[1]$ e $p[1]/s[2]$. La differenza totale è $|10-9|+|14-5| = 10$. Se l'associazione fosse $p[1]/s[1]$ e $p[2]/s[2]$, la differenza totale sarebbe: $|9-5|+|10-14| = 8$.

Esercizio 4

Scrivere le ricorrenze di Programmazione Dinamica che, date due sequenze $X = x_1, x_2, \dots, x_n$ e $Y = y_1, y_2, \dots, y_m$ di caratteri e un numero intero (positivo) k , dicano se esiste una sottosequenza comune (LCS) di X e Y di lunghezza maggiore o uguale a k .

Suggerimento Si utilizzi un solo coefficiente a valori booleani, che vale **true** se esiste una LCS di lunghezza maggiore o uguale di una lunghezza fissata, **false** altrimenti.

Soluzione

$$LCS(X(m), Y(n), k) = \begin{cases} \text{false} & k > 0 \wedge (m = 0 \vee n = 0) \\ \text{true} & k = 0 \\ LCS(X(m-1), Y(n-1), k-1) & x_m = y_n \wedge m > 0 \wedge n > 0 \\ LCS(X(m-1), Y, k) \vee LCS(X, Y(n-1), k) & x_m \neq y_n \wedge m > 0 \wedge n > 0 \end{cases}$$

Algoritmi e Strutture Dati - Prova d'esame

29/01/07

Esercizio 1 - Stessa distanza da nodi

Si consideri il seguente problema: dato un grafo orientato G (con n nodi e m archi) e due nodi s_1 e s_2 di G , trovare il numero di nodi di G raggiungibili da s_1 che si trovano alla stessa distanza da s_1 e da s_2 .

a) Scrivere in pseudocodice (commentato!) un algoritmo in grado di risolvere questo problema e descriverne brevemente il funzionamento.

b) Discutere la complessità dell'algoritmo proposto.

Ricordiamo che in un grafo orientato G , dati due nodi s e v , si dice che v è raggiungibile da s se esiste un cammino da s a v e, in questo caso, la distanza di v da s è la lunghezza del più corto cammino da s a v .

Soluzione

L'algoritmo esegue una esplorazione in ampiezza del grafo diretto G a partire dal nodo s_1 calcolando così le distanze di tutti i nodi da s_1 . Tali distanze sono memorizzate in un vettore A che ha per indici i nodi del grafo. Successivamente con una analoga visita in ampiezza si determinano le distanze dei nodi da s_2 calcolando un vettore delle distanze B . Infine l'algoritmo confronta le componenti corrispondenti dei vettori A e B contando il numero dei nodi che hanno uguale valore.

L'algoritmo è descritto dal seguente schema nel quale si utilizza la procedura $\text{Ampiezza}(G, v)$ che restituisce il vettore D delle distanze dei nodi di G dal vertice v . Per convenzione, $D[w]$ risulterà uguale a -1 se e solo se il nodo w non è raggiungibile da v , altrimenti $D[w]$ sarà proprio la distanza di w da v .

Algorithm 1 STESSADISTANZA(G, s_1, s_2)

```
1:  $A \leftarrow \text{Ampiezza}(G, s_1)$ 
2:  $B \leftarrow \text{Ampiezza}(G, s_2)$ 
3:  $R \leftarrow 0$ 
4: for all  $w \in V$  do
5:   if  $A[w] \geq 0 \wedge A[w] = B[w]$  then  $R \leftarrow R + 1$ 
6: return  $R$ 
```

Il calcolo delle distanze si ottiene applicando la seguente procedura che esegue un attraversamento in ampiezza. Il procedimento è basato sull'utilizzo della coda Q . Per ogni v , $L(v)$ è la lista dei nodi w per i quali esiste in G l'arco (v, w) , ovvero è la lista di adiacenza.

Algorithm 2 AMPIEZZA(G, v)

```
1:  $Q = \text{empty queue}$ 
2:  $Q \leftarrow \text{Enqueue}(Q, v)$ 
3: for all  $w \in V$  do
4:    $D[w] \leftarrow -1$ 
5:  $D[v] \leftarrow 0$ 
6: while  $Q$  is not empty do
7:    $w \leftarrow \text{Front}(Q)$ 
8:    $Q \leftarrow \text{Dequeue}(Q)$ 
9:   for all  $z \in L(w)$  do
10:    if  $D[z] == -1$  then
11:       $D[z] := D[w] + 1$ 
12:       $Q := \text{Enqueue}(Q, z)$ 
return  $D$ 
```

Esercizio 2 - Foglie con costo minimo

Dato un albero con radice T , con insieme dei nodi V , e una funzione peso $p : V \rightarrow \mathbb{R}$, chiamiamo costo di un nodo $v \in V$ il valore

$$c(v) = \begin{cases} p(v) & \text{se } v \text{ è la radice} \\ p(v) + c(\text{padre}(v)) & \text{altrimenti} \end{cases}$$

Dato un albero con radice T , rappresentato mediante liste di adiacenza, e per ogni nodo v di T il peso $p(v) \in \mathbb{R}$, scrivere un algoritmo in pseudocodice commentato che ritorni l'insieme delle foglie di T che hanno costo minimo.

Soluzione

Il problema può essere risolto con una visita in preordine dell'albero T nella quale, per ogni nodo v diverso dalla radice, il valore $c(v)$ viene calcolato conoscendo il costo del padre di v . Descriviamo quindi un programma principale che richiama una procedura ricorsiva, chiamata $Visita(v)$, che calcola i valori $c(v)$ di ogni nodo v diverso dalla radice. Tale procedura calcola anche una lista F contenente le foglie di T accompagnate dal relativo costo e determina il valore minimo m di tali costi. Il programma principale poi scorrerà la lista F e determinerà l'elenco delle foglie che hanno costo m . Nella sua computazione la chiamata $Visita(v)$ mantiene, oltre alle variabili globali F e m , anche la variabile globale C che rappresenta il costo $c(v)$ del nodo corrente v . Nella descrizione che segue supponiamo che V sia l'insieme dei nodi di T , r sia la radice di T e, per ogni nodo $v \in V$, $L(v)$ sia la lista dei figli di v .

Algorithm 3 MAIN()

```

1:  $m \leftarrow \infty$ 
2:  $F \leftarrow$  empty list
3:  $C \leftarrow p(r)$ 
4:  $Visita(r)$ 
5:  $S \leftarrow$  empty list
6: for all  $(v, x) \in F$  do
7:   if  $x = m$  then  $S \leftarrow$  Inserisci in testa( $S, v$ )
8: return  $S$ 
```

Algorithm 4 VISITA(V)

```

1: if  $L(v) ==$  empty (ovvero  $v$  è una foglia) then
2:    $F :=$  Inserisci in testa( $F, (v, C)$ )
3:   if  $C < m$  then
4:      $m \leftarrow C$ 
5: else
6:   for all  $w \in L(v)$  do
7:      $C \leftarrow C + p(w)$ 
8:      $Visita(w)$ 
9:    $C \leftarrow C - p(v)$ 
```

Esercizio 3 - Ricorrenza

Risolvere, spiegando i passaggi, la seguente ricorrenza:

$$T(n) = \begin{cases} \sqrt{n}T(\sqrt{n}) + an & n > 2 \\ 1 & \text{altrimenti} \end{cases}$$

con $a > 0$.

Suggerimenti: Effettuare un cambio di variabile per ricondursi ad una ricorrenza conosciuta. Ignorare il fatto che $\sqrt{n}$ possa non essere intero.

Soluzione

Poniamo $n = 2^k$ (ovvero $k = \log n$) nella ricorrenza e otteniamo:

$$\begin{aligned} T(n) &= \sqrt{n}T(\sqrt{n}) + an \Rightarrow \\ T(2^k) &= 2^{\frac{k}{2}}T(2^{\frac{k}{2}}) + a2^k \end{aligned}$$

Dividiamo tutto per 2^k , e otteniamo:

$$\frac{T(2^k)}{2^k} \leq \frac{T(2^{\frac{k}{2}})}{2^{\frac{k}{2}}} + a$$

Poniamo ora $S(k) = \frac{T(2^k)}{2^k}$; otteniamo quindi:

$$S(k) = S(k/2) + a$$

A questo punto, possiamo utilizzare il Master Theorem. Siccome ricadiamo nel caso 2, otteniamo che

$$S(k) = \Theta(\log k)$$

A questo punto, possiamo tornare indietro e ottenere:

$$\begin{aligned}\frac{T(2^k)}{2^k} &= \Theta(\log k) \Rightarrow \\ T(2^k) &= \Theta(\log k)2^k \Rightarrow \\ T(n) &= \Theta(\log \log n)n \Rightarrow \\ T(n) &= \Theta(n \log \log n)\end{aligned}$$

Esercizio 4 - somma di interi divide et impera

Si consideri il seguente problema: data una sequenza di $n \geq 1$ numeri interi $a_1, a_2, \dots, a_n$, trovare la somma $b = a_1 + a_2 + \dots + a_n$.

a) Scrivere in pseudocodice (commentato!) un algoritmo del tipo “divide et impera” in grado di risolvere questo problema e descriverne brevemente il funzionamento.

b) Discutere la complessità dell’algoritmo proposto.

Nota bene: l’algoritmo deve essere del tipo “divide et impera”.

Soluzione

L’idea dell’algoritmo è quella di applicare una procedura ricorsiva che spezza l’input in due parti (circa) uguali, richiama se’ stessa ricorsivamente su queste ultime e quindi restituisce la somma dei due risultati ottenuti. Formalmente l’algoritmo si riduce alla chiamata

$$C := Somma(1, n)$$

della procedura Somma(i, j), definita nel seguito, che restituisce la somma degli elementi

$$a_i, a_{i+1}, \dots, a_j$$

con $1 \leq i \leq j \leq n$.

Algorithm 5 SOMMA(i, j)

```
1: if  $i == j$  then
2:   return  $a[i]$ 
3:  $k \leftarrow \lfloor \frac{j+i}{2} \rfloor$ 
4:  $b \leftarrow Somma(i, k)$ 
5:  $c \leftarrow Somma(k+1, j)$ 
6: return  $b + c$ 
```

Algoritmi e Strutture Dati - Prova d'esame

08/06/07

Esercizio 1 - Quando Huffman non serve....

Si consideri un file di testo di lunghezza $n = kd$, in cui compaiono esattamente k caratteri distinti. Ogni carattere compare un numero eguale di volte pari a $d = n/k$.

Secondo la notazione ASCII, il numero totale di bit necessari per rappresentare questo testo è $8n$. Si consideri invece la rappresentazione tramite codice di Huffman.

- Quanti bit $len(k, n)$ sono necessari per rappresentare il testo?
- Per quali valori di k si ha che $len(k, n) \leq 8n$?

In entrambi i casi, spiegare nei dettagli come si sono ottenute le formule.

Soluzione Per avere un'idea dell'albero per la decodifica che si ottiene, si consideri un alfabeto composto dai caratteri $\{0, \dots, 9\}$, ognuno dei quali ha una frequenza pari al 10%.

L'albero ottenuto è mostrato in figura 1

I codici associati ai caratteri sono i seguenti:

9 -> 1
8 -> 01
7 -> 001
6 -> 0001
5 -> 00001
4 -> 000001
3 -> 0000001
2 -> 00000001
1 -> 000000000
0 -> 0000000001

Il valore di $len(k, n)$ è pari a:

$$\begin{aligned} len(k, n) &= \left(\sum_{i=1}^{k-1} i \cdot \frac{n}{k} \right) + (k-1) \cdot \frac{n}{k} \\ &= \left(k \frac{(k-1)}{2} + k-1 \right) \frac{n}{k} \\ &= n \cdot \frac{k^2 + k - 2}{2k} \end{aligned}$$

Per rispondere alla seconda domanda, dobbiamo soddisfare la seguente disequazione.

$$\begin{aligned} n \cdot \frac{k^2 + k - 2}{2k} &\leq 8n \\ \Leftrightarrow k^2 + k - 2 &\leq 16k \\ \Leftrightarrow k^2 - 15k - 2 &\leq 0 \end{aligned}$$

Utilizzando la soluzione delle equazioni di secondo grado:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

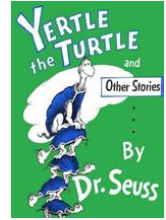
otteniamo gli zeri: $15.13\dots$ e $-0.13\dots$. Poichè la disequazione è negativa fra le due soluzioni, il nostro codice è più efficiente nel range $k \in [1 \dots 15]$.

Esercizio 2 - Yertle the Turtle

Una storia per bambini pubblicata nel 1958 (Yertle the Turtle ¹) parla di una trono fatto di tartarughe impilate. Il nostro compito è impilare una serie di tartarughe in modo tale che i gusci non si rompano.

Ogni tartaruga i ha un peso $p[i]$ e un resistenza $f[i]$, che rappresenta la sua capacità di carico (incluso il proprio peso). In altre parole, se una tartaruga ha resistenza 1000 e peso 300, tartarughe di un peso complessivo fino a $1000 - 300 = 700$ possono essere impilate sopra di essa.

Scrivere un programma che, preso in input due array p e f di lunghezza n , riportino in output il numero massimo di tartarughe che possono essere impilate. Discutere la correttezza e la complessità dell'algoritmo.



Soluzione Questo problema può essere risolto tramite programmazione dinamica.

Definiamo innanzitutto l'array di resistenza residua r , dove $r[i] = f[i] - p[i]$ rappresenta il peso totale delle tartarughe che possono essere caricate sopra la tartaruga i .

Dimostriamo formalmente che il problema ha **sottostruttura ottima**. Sia $P(i, t)$ il problema di individuare la più lunga pila di tartarughe, prese dall'intervallo $i \dots n$, in modo che il loro peso complessivo sia inferiore o uguale a t . Sia $A(i, t) = a_1 \dots a_n$ una soluzione ottima di $P(i, t)$, dove $a_1 \dots a_k$ sono gli indici delle tartarughe scelte, dalla tartaruga più in basso a quella più in alto. E' semplice vedere che $A(i + 1, \min\{t - p[a_1], r[a_1]\}) = a_2 \dots a_n$ è una soluzione ottima di $P(i + 1, \min\{t - p[a_1], r[a_1]\})$; se così non fosse, e ci fosse una sequenza $A'(i + 1, \min\{t - p[a_1], r[a_1]\}) = a'_2 \dots a'_k$ più lunga di $A(i + 1, \min\{t - p[a_1], r[a_1]\})$, basterebbe sostituire questa in $A(i, t)$ e otterremo una soluzione migliore di $A(i, t)$, assurdo per definizione.

Definiamo l'array $c[n, t]$ contenente la dimensione della soluzione ottima di $P(n, t)$. Tale array può essere calcolato nel modo seguente:

$$c[n, t] = \begin{cases} 1 & i == n \wedge p[i] \leq t \\ 0 & i == n \wedge p[i] > t \\ c[i + 1, t] & p[i] > t \\ \max\{c[i + 1, t], c[i + 1, \min\{t - p[a_1], r[a_1]\}]\} & \text{altrimenti} \end{cases}$$

Il problema iniziale corrisponde a $\text{count}(1, \max\{r[i]\})$.

Il codice segue dalla definizione della tabella $c[n, t]$, che può essere calcolata tramite memoization oppure con programmazione dinamica a partire dalla riga n .

La complessità è elevata, $O(nk)$ dove k è la massima resistenza residua fra le n tartarughe.

Una versione alternativa, con complessità $O(n^2)$ può essere la seguente: si sceglie la tartaruga con la maggiore resistenza residua e si mette in fondo alla pila. Si scorrono tutte le tartarughe e si cerca, fra quelle con peso inferiore alla resistenza residua, quella che produce una resistenza residua maggiore.

```
// Inizializza r
for i := 1 to n do r[i] = f[i]-p[i]

// Cerca la tart. con resistenza residua massima
// la conta e la elimina dai prossimi giri
max := 1
for i := 2 to n do
    if (r[i] > r[max]) then
        max := i
residual = r[max]
r[max] = -1
count := 1

// Cerca di aggiungere tartarughe; smette non appena
// non e' piu' possibile aggiungerne
do
    maxr := -1
    for i := 1 to n do
        if (r[i] >= 0 AND p[i] <= residual) then
            newr = min(residual-p[i], r[i])
```

¹http://en.wikipedia.org/wiki/Yertle_the_Turtle

```

    if (newr > maxr) then
        maxr = newr
        maxi = i
    if (maxr > 0)
        count := count+1
        residual := maxr
        r[i] := -1
    endif
while (maxr > 0)
return count

```

Si noti che è possibile migliorare questo algoritmo eliminando dall'array le tartarughe il cui peso è superiore al residuo; qui invece tutte le tartarughe vengono considerate ad ogni ciclo while. Questo è un'ottimizzazione che non cambia la complessità nel caso pessimo. L'eliminazione può essere effettuata sovrascrivendo l'elemento da eliminare con quello in posizione n , e diminuendo n di 1.

Esercizio 3 - Sequenze 5-crescenti

Una sequenza di interi $x_1, x_2, \dots, x_n$ si dice 5-crescente se risulta vero che per ogni $j > i + 5$ si ha che $x_j > x_i$. Si analizzi la complessità del caso peggiore dei seguenti algoritmi di ordinamento quando l'input è sempre una sequenza 5-crescente:

1. MergeSort: $\Theta(n)$, $\Theta(n \log n)$ oppure $\Theta(n^2)$
2. QuickSort: $\Theta(n)$, $\Theta(n \log n)$ oppure $\Theta(n^2)$

Soluzione Premessa: tutti questi algoritmi di ordinamento hanno complessità $\Omega(n \log n)$ quindi la risposta $\Theta(n)$ è sempre sbagliata.

1. MergeSort: questo algoritmo impiega sempre tempo $\Theta(n \log n)$ su qualsiasi sequenza di input. Quindi il fatto che l'input sia 5-crescente è irrilevante. Pertanto il caso pessimo è $\Theta(n \log n)$ e non differisce dal caso generale.
2. QuickSort: Il caso pessimo di Quicksort è $\Theta(n^2)$ quando l'input è già ordinato. Una sequenza crescente è anche 5-crescente per cui il fatto che si limiti l'input a sequenze 5-crescenti non modifica il caso pessimo ottenuto nel caso di input generale. Pertanto in questo caso la risposta è $\Theta(n^2)$ e non differisce dal caso generale.

Esercizio 4

Trovare un limite asintotico superiore e un limite asintotico inferiore alla seguente ricorrenza, facendo uso del metodo di sostituzione:

$$T(n) = 2T(n/8) + 2T(n/4) + n$$

Soluzione E' facile dimostrare che ovviamente, la funzione $T(n)$ è $\Omega(n)$; infatti,

$$T(n) = 2T(n/8) + 2T(n/4) + n \geq n \geq c_1 n$$

per $c_1 \leq 1$. Non è necessario dimostrare il caso base, perchè abbiamo rimosso gli elementi ricorsivi e quindi non abbiamo bisogno di induzione.

Una prima osservazione che si potrebbe fare per "indovinare" un limite superiore è la seguente:

$$\begin{aligned}
 T(N) &= 2T(n/8) + 2T(n/4) + n \\
 &\leq 2T(n/4) + 2T(n/4) + n \\
 &\leq 4T(n/4) + n
 \end{aligned}$$

In base a questo risultato, il master theorem dice che $T(n) = O(n \log n)$. E' facile dimostrare questo risultato. A questo punto, non ci resta che vedere quale dei due risultati ($\Omega(n)$ e $O(n \log n)$) è stretto. Proviamo con $O(n)$.

Ipotesi induttiva: $\forall n < n' : T(n) \leq cn$. Proviamo che il risultato è valido anche per n .

$$\begin{aligned} T(n) &= 2T(n/8) + 2T(n/4) + n \\ &\leq 2cn/8 + 2cn/4 + n \\ &= 3/4cn + n \\ &\leq cn \end{aligned}$$

L'ultima disequazione è vera se $3/4c + 1 \leq c$, ovvero se $c \geq 4$.

Quindi la nostra funzione è $\Theta(n)$.

Algoritmi e Strutture Dati - Prova d'esame

23/07/07

Esercizio 1

Scrivere un algoritmo per trovare la “moda” di un insieme di dati (ovvero il valore che compare più spesso). Nel caso di più valori che possono essere definiti come moda, ritornare il valore numericamente più alto.

Ad esempio, nell’array

1 2 7 12 3 6 7 18 28 11 28 11 12 12 14 3 5 8 9 2 38 28

la moda è 28.

Soluzione

```
int moda(A)
  sort(A)
  max = 0
  moda = ?
  count = 1
  val = A[1]
  for i:= 2 to A.length do
    if (A[i] == val)
      count = count+1
    else
      if (count >= max)
        max = count
        moda = val
      endif
      count = 1
      val = A[i]
    endif
  endfor
```

Esercizio 2

Dimostrate che per ogni costante $a > 0$, ogni costante $b > 0$, $(n + a)^b = \Theta(n^b)$.

Soluzione Per dimostrare che $(n + a)^b = \Theta(n^b)$, dobbiamo trovare tre costanti $c_1, c_2, n_0 > 0$ tali che

$$0 \leq c_1 n^b \leq (n + a)^b \leq c_2 n^b, \forall n \geq n_0$$

Si noti che $n + a \leq 2n$, per $a \leq n$; e che $n + a \geq \frac{1}{2}n$, per qualsiasi n . Quindi, per $n \geq a$,

$$0 \leq \frac{1}{2}n \leq n + a \leq 2n$$

Poichè $b > 0$, si ha che

$$0 \leq \frac{1}{2}^b n^b \leq (n + a)^b \leq 2^b n^b$$

quindi $c_1 = \frac{1}{2}^b, c_2 = 2^b$ e $n_0 = a$.

Esercizio 3

Siano dati due alberi di ricerca T_1 e T_2 , rispettivamente con n_1 e n_2 nodi. Scrivere un algoritmo di merging dei due alberi, che restituisca un unico albero binario di ricerca di dimensione $n_1 + n_2$, in tempo $O(n_1 + n_2)$.

Soluzione

Soluzione errata Nel dare l'esercizio, ho erroneamente "assunto" una soluzione facile, basata sulla procedura `successor()` come definita dai lucidi e sfruttando gli stessi principi della funzione `merge()` di Mergesort. Dimenticando però che l'inserimento ordinato in un ABR produce un albero completamente sbilanciato, con un costo di inserimento totale $O((n_1 + n_2)^2)$. Visto il mio errore, ho comunque considerate valide soluzioni di questo tipo. Di seguito la soluzione sbagliata:

```
ABR merge (ABR T1, ABR T2)
    Ti = T1.minimum()
    Tj = T2.minimum()
    T = new ABR()
    while (Ti != null and Tj != null) do
        if (Ti.key() < Tj.key())
            T.insert(Ti.key(), Ti.data())
            Ti = Ti.successor()
        else
            T.insert(Tj.key(), Tj.data())
            Tj = Tj.successor()
        endif
    endwhile
    while (Ti != null and Tj == null) do
        T.insert(Ti.key(), Ti.data())
        Ti = Ti.successor()
    endwhile
    while (Ti == null and Tj != null) do
        T.insert(Tj.key(), Tj.data())
        Tj = Tj.successor()
    endwhile
    return T
```

Soluzione corretta L'idea di funzionamento è semplice: prima si crea un array di appoggio contenente tutti i valori ordinati. Questo può essere fatto in tempo $\Theta(n_1 + n_2)$, utilizzando la stessa tecnica di cui sopra.

A questo punto, bisogna inserire i vari dati in un albero ABR. E' una buona pratica garantire che l'albero così risultante sia bilanciato. E' possibile utilizzare una tecnica divide et impera per ottenere tale risultato.

Si prende l'array $A[k \dots j]$; si calcola l'indice mediano $m = (k + j)/2$; il valore nella posizione $A[m]$ è la radice del nostro albero. Si divide l'array in due sottoparti di dimensione approssimativamente uguali, $A[k \dots m - 1]$ e $A[m + 1 \dots j]$; si applica ricorsivamente l'algoritmo, ottenendo due sottoalberi T_l e T_r , rispettivamente il figlio destro e il figlio sinistro di T .

La funzione di ricorrenza associata a questa procedura è:

$$T(n) = 2T(n/2) + O(1)$$

Utilizzando il Master Theorem, otteniamo una complessità di $\Theta(n) = O(n_1 + n_2)$.

```
ABR merge (ABR T1, ABR T2)
    Ti = T1.minimum()
    Tj = T2.minimum()
    int k = 1
    while (Ti != null and Tj != null) do
        if (Ti.key() < Tj.key())
            keys[k] = Ti.key()
            data[k] = Ti.data()
            Ti = Ti.successor()
        else
            keys[k] = Tj.key()
            data[k] = Tj.data()
            Tj = Tj.successor()
        endif
    endwhile
    while (Ti != null and Tj == null) do
        keys[k] = Ti.key()
        data[k] = Ti.data()
        Ti = Ti.successor()
    endwhile
```

```

endwhile
while (Ti == null and Tj != null) do
    keys[k] = Tj.key()
    data[k] = Tj.data()
    Tj = Tj.successor()
endwhile
return buildTree(keys, data, 1, k)

ABD buildTree(keys, data, k, j)
    m = (k+j)/2
    T = new ABR(keys[m], data[m]);
    T.left = buildTree(keys, data, k, m-1)
    T.right = buildTree(keys, data, m+1, k)
    return T

```

Esercizio 4

Il conduttore di una radio deve ogni giorno compilare il palinsesto del suo programma musicale, prevedendo la programmazione di alcune delle canzoni dalla lista di canzoni della stazione, indicizzate con i numeri da 1 a n , in modo da massimizzare il gradimento dei suoi ascoltatori. La radio lancia ogni giorno un sondaggio col quale gli ascoltatori possono votare le loro canzoni preferite. Il conduttore dispone quindi della funzione $g[i], i = 1 \dots n$, che contiene il risultato del giorno precedente, in forma di valori da 0 (nessun gradimento) ad 1 (massimo gradimento). Il programma radiofonico ha la durata di N minuti e ciascuna canzone ha la sua durata $d[i], i = 1 \dots n$. Definire un algoritmo che presi i dati descritti sopra, fornisca in uscita il palinsesto per il programma che massimizza il gradimento del pubblico (nota che il palinsesto non pu contenere piu occorrenze della stessa canzone).

Soluzione Questo è un problema di zaino 0-1. Basta utilizzare l'algoritmo basato su programmazione dinamica visto a lezione (non lo ripeto qui). Nota: nonostante il gradimento sia un valore nell'intervallo $[0, 1]$, questo non è uno zaino frazionario: infatti le canzoni non possono essere suonate parzialmente.

Algoritmi e Strutture Dati - Prova d'esame

27/08/07

Esercizio 1

- Descrivere un algoritmo per risolvere il seguente problema. Dato un albero binario T di n nodi rappresentato tramite puntatori `left` e `right`, e un intero k , $0 \leq k \leq n - 1$, trovare il numero di nodi di T che hanno altezza k .
- Valutare in funzione di n il tempo di calcolo richiesto dall'algoritmo nel caso peggiore.

Soluzione Ricordiamo che in un albero binario T , l'altezza di un nodo v è la massima distanza di v da una foglia. Vogliamo descrivere un algoritmo che calcoli l'altezza di v , per ogni nodo v dell'albero T . E' chiaro che l'altezza di una foglia è; invece, l'altezza di un nodo interno v si calcola facilmente una volta note le altezze dei suoi figli: è sufficiente considerare la massima tra queste e incrementarla di 1. Quindi, per risolvere il nostro problema è sufficiente descrivere un algoritmo che esegue una visita di T in ordine posticipato. Definiamo una procedura ricorsiva `visita(v)` che, su input v , restituisce l'altezza di v . Tale procedura restituisce il valore 0 se v è una foglia; altrimenti richiama se stessa sui figli di v determinando il valore massimo ottenuto, incrementa di 1 tale valore e lo assegna alla variabile d . Se inoltre d coincide con k si incrementa un opportuno contatore rappresentato da una variabile globale S (inizialmente posta a 0). Il valore finale di S sarà l'output dell'algoritmo. Il costo della visita postordine è $O(n)$.

```
S := 0
visita(r)
return S

visita(v)
  if v.left = nul then
    a := -1
  else
    a := visita(v.left)
  if v.right = nul then
    b := -1
  else
    b := visita(v.right)
  if a <= b then
    d = b+1
  else
    d = a+1
  if d=k then
    S := S+1
  return d
```

Esercizio 2

Sia $G = (V, E)$ un grafo non orientato connesso e sia s un nodo in V . Si ricorda che la distanza da s ad un nodo $v \in V$ è pari al numero minimo di archi che separa s da v . Scrivere un algoritmo che ritorni la distanza media da s di tutti gli altri nodi del grafo (quindi escluso s).

Nell'esempio della figura 1, la distanza media di b è 1, mentre la distanza media di a e c è 1.5.

Soluzione

```
sum-distance(G, s)
  for each v in V do v.dist := -1
  F := new Queue()
  F.enqueue(s)
  s.dist := 0
  tot := 0
  count := -1
  while F <> empty do
```

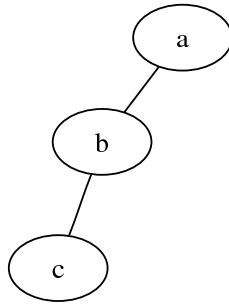


Figura 1: Esempio

```
u = F.dequeue()
tot := tot + u.dist
count := count + 1
for each v in u.adj do
  if (v.dist < 0) then
    v.dist = u.dist+1
    F.enqueue(v)
return tot/count
```

Esercizio 3

Una rete telematica è formata da vari nodi di smistamento e varie connessioni tra i nodi. Ogni connessione ha una capacità limitata (per esempio una connessione modem può essere limitata da 56Kb/sec). Il nostro obiettivo è inviare n filmati *distinti* $f_1 \dots f_n$ da un computer “server” s a n computer “client” $c_1, \dots, c_n$; tutti i filmati richiedono t bit/s per essere trasmessi. Si proponga un algoritmo per stabilire se la rete a nostra disposizione è in grado di trasmettere i filmati alle destinazioni.

Soluzione Si utilizzano gli algoritmi di flusso massimo. La rete di connessione viene rappresentata da un grafo, con gli archi pesati dalla capacità limitata. Il nodo server costituisce la sorgente; si aggiunge un superpozzo, e un arco con peso k da ognuno dei nodi client al superpozzo. Si esegue un algoritmo di calcolo del flusso massimo, e se il flusso massimo è uguale a tn , la rete è in grado di trasmettere tutti i filmati.

Esercizio 4

Soluzione

Algoritmi e Strutture Dati - Prova d'esame

20/12/07

Esercizio 1

Con riferimento all'algoritmo MISTERO1,

- comprendere e spiegare il funzionamento dell'algoritmo
- trovare limite asintotico superiore più stretto possibile in funzione della dimensione dell'input, utilizzando il metodo di sostituzione.

Algorithm 1 MISTERO1(A, k)

1: return mistero2($A, 0, A.length - 1, k$)

Algorithm 2 MISTERO2($A, first, last, k$)

1: if($first > last$) return -1
2: if($A[first] == k$) return first
3: if($A[last] == k$) return last
4: return mistero2($arr, first + 1, last - 1, k$)

Soluzione La funzione MISTERO1 esegue di fatto una scansione dell'array A , cercandovi l'intero k . Se k è presente ne restituisce la posizione, altrimenti restituisce -1 . La scansione avviene controllando la prima e l'ultima posizione dell'array, quindi, ricorsivamente, cercando k su una porzione più ristretta dell'array stesso. Ciò è in effetti svolto dalla funzione MISTERO2 che cerca l'intero k nell'array A , a partire dalla posizione $first$, fino alla posizione $last$. Quando la porzione di array su cui si fa ricerca si annulla ($first > last$) la ricerca termina con insuccesso.

Il costo $T_1(n)$ di MISTERO1 dipende strettamente dal costo $T_2(n)$ di MISTERO2: $T_1(n+1) = T_2(n+3)$, ovvero $T_1(n) = T_2(n+2)$. Per quanto riguarda T_2 possiamo scrivere la seguente equazione di ricorrenza:

$$T_2(n) = \begin{cases} O(1) & \text{if } x = 1 \\ O(1) & \text{if } x = 2 \\ T_2(n-1) + O(1) & \text{if } x > 2 \end{cases}$$

Proviamo a dimostrare che $T_2(n) = O(n)$. Utilizziamo il metodo di sostituzione. La nostra ipotesi induttiva è che $T_2(m) \leq cm$ per tutti gli $m < n$, e vogliamo dimostrare che la proprietà è vera per n .

$$\begin{aligned} T_2(n) &= T_2(n-2) + d \\ &\leq c(n-2) + d \\ &= cn - 2c + d \end{aligned}$$

Affinché ciò sia $\leq cn$ dobbiamo avere $c \geq d/2$.

Come caso base, possiamo prendere $T_2(1) = e \leq c$ e $T_2(2) = f \leq 2c$.

Possiamo scegliere $c \geq \max(d/2, e, f/2)$.

Quindi $T_2(n) = O(n)$ e di conseguenza $T_1(n) = O(n)$

Esercizio 2

Dato in input un array di n numeri compresi fra 1 e M , progettare un algoritmo che ritorna vero se esiste un sottoinsieme di essi la cui somma è uguale a C . Spiegare il funzionamento dell'algoritmo nella maniera più formale possibile e discutere la complessità dell'algoritmo risultante.

Soluzione Il problema può essere risolto tramite programmazione dinamica considerandolo un problema di zaino in cui il peso coincide con il valore.

Esercizio 3

Gestire un mcdonald non e' semplice. La giornata lavorativa e' suddivisa in 3 turni da quattro ore, dalle 11 alle 23. Il management ha stabilito che per ognuno dei turni contenuti nell'insieme T (21 turni totali), una certa quantita' di personale e' necessaria. Ad esempio, sabato dalle 19 alle 23 c'e' la richiesta massima. Dato il turno $t_i \in T$, la richiesta di personale e' p_i . Avete a disposizione un insieme D di dipendenti; ogni dipendente $d_j \in D$ ha un insieme di turni $n_j \subseteq T$ in cui non puo' lavorare. Ad esempio, d_j non puo' lavorare nei turni $\{t_1, t_7, t_{21}\}$. Per contratto aziendale, ogni lavoratore non puo' lavorare per piu' di cinque turni. Ogni turno t_i deve essere coperto esattamente da p_i personale. Ogni giorno, un dipendente non puo' lavorare per piu' di due turni (qualsiasi, anche non consecutivi). Progettare un algoritmo che produca uno scheduling che illustri, per ogni turno, il personale associato. Descrivere l'algoritmo nella maniera piu' formale possibile e discutere la complessita' dell'algoritmo risultante.

Soluzione Il problema puo' essere ridotto ad un problema di flusso massimo. A questo scopo, e' necessario costruire un grafo a partire dai dati di input.

La Figura 1 contiene tale grafo; per ragioni di spazio, sono rappresentati solo due giorni della settimana.

Il grafo e' cosi' composto: per ogni dipendente d_j , aggiungiamo un nodo D_j . Per ogni turno t_i , aggiungiamo un nodo $T_{d,h}$, dove d e' il giorno della settimana e h e' l'ora di inizio turno. Per ogni coppia (dipendente D_j , giorno d) aggiungiamo un nodo $W_{j,d}$. Infine, aggiungiamo la sorgente S e il pozzo T .

Poiche' ogni dipendente puo' lavorare al massimo 5 turni, mettiamo il peso 5 su tutti gli archi (S, D_j) . Poiche' ogni dipendente puo' lavorare al massimo 2 turni al giorno, mettiamo il peso 2 su tutti gli archi $(D_j, W_{j,d})$. Le preferenze dei dipendenti sono rappresentate dagli archi $(W_{j,d}, T_{d,h})$, tutti con peso 1. Infine, si collegano i turni al nodo pozzo, con peso pare a p_i .

Esiste un'assegnazione dei dipendenti sui turni se e solo se il flusso e' esattamente pari alla somma dei pesi $P = \sum p_i$. Il costo computazionale dell'algoritmo e' $O(P(V + E))$. Poiche' ogni lavoratore puo' lavorare al massimo 5 turni, allora $P \leq 5|D|$. V e' pari a $8|D| + 25$ ed E e' $O(29|D| + 21)$, quindi il costo totale e' $O(D^2)$.

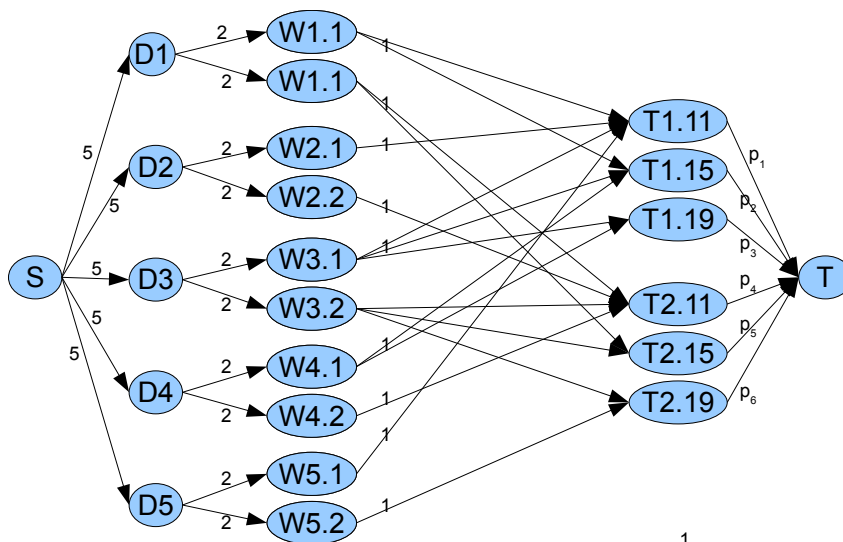


Figura 1: McDonald

Esercizio 4

Sia dato un grafo non orientato $G = (V, E)$ e una funzione di pesi $w : E \rightarrow R$. Siano e_1 l'arco di peso minimo e e_2 il secondo arco di peso minimo.

Confermare o provare la seguente affermazione: entrambi gli archi e_1 ed e_2 fanno parte dello stesso albero di copertura minimo.

Soluzione Utilizzando l'algoritmo di Kruskal, e' ovvio che l'arco di peso minimo (u, v) fa parte di un MST. Ma anche il secondo ne fa parte, in quanto non possono esserci due archi che insistono sulla stessa coppia di nodi, e dopo avere inserito (u, v) l'unico insieme congiunto contiene solo due nodi.

Algoritmi e Strutture Dati - Prova d'esame

07/01/08

Esercizio 1

Siete i sistemisti di un data center in grado di “macinare” diversi terabyte di dati al giorno. In un periodo di n giorni, vi viene comunicato in anticipo la quantità x_i di dati da gestire nel giorno i . I dati non macinati nel giorno i non possono essere gestiti il giorno successivo. Purtroppo, la scelta del sistema operativo non è stata fra le più felici. Infatti le performance del sistema peggiorano di giorno in giorno, fino a quando non decidete di fare reboot (forse per memory leaks?). Dopo j giorni dall'ultimo reboot, la quantità massima di terabyte “macinabili” è pari a s_j , con $s_1 > s_2 > \dots > s_n$. Peggio ancora, per fare reboot ci vuole un giorno intero e durante quel giorno non è possibile gestire alcun dato.

Dato un array $x[1 \dots n]$ che misura la quantità di dati in input e un array $s[1 \dots n]$ che misura le performance di sistema dall'ultimo reboot, scrivere un programma che calcola il numero massimo di terabyte che possono essere processati dal sistema, e valutarne la sua complessità.

Esempio:

i	1	2	3	4
x	10	1	7	7
s	8	4	2	1

La soluzione ottimale consiste nel fare un singolo reboot nel giorno 2. Si processano quindi 8TB nel giorno 1, 7TB nel giorno 3, 4TB nel giorno 4 per un totale di 19TB.

Soluzione Il problema può essere risolto con la programmazione dinamica. Sia $P(i, j)$ la massima quantità di terabyte che si possono gestire nei giorni $[i, \dots, n]$, nel caso in cui l'ultimo reboot sia stato effettuato j giorni prima del giorno i . Il problema originale corrisponde a calcolare $P(1, 1)$.

Si può facilmente vedere che $P(i, j)$ può essere calcolato ricorsivamente come segue:

$$P(i, j) = \begin{cases} \min\{x_i, s_j\} & i = n \\ \max\{P(i+1, 1), \min\{x_i, s_j\} + P(i+1, j+1)\} & i < n \wedge j < n \end{cases}$$

Il codice che ne risulta è il seguente:

```
datacenter(x[1..n], s[1..n])
  for j := 1 to n do
    P[n, j] = min { x[i], s[j] }
  for i := n-1 to 1 do
    for j := 1 to n-1 do
      P[i, j] = max ( P[i+1, 1], min ( x[i], s[j] ) + P[i+1, j+1] )
  return P[1, 1]
```

Tale programma lavora in tempo $O(n^2)$.

Esercizio 2

Siete stati incaricati di redigere i piani di evacuazione di una regione minacciata da un vulcano. In questa regione, che può essere rappresentata da un insieme di nodi V e un insieme di archi (strade) E che connettono i nodi. Esiste un sottoinsieme $P \subset V$ di nodi detti villaggi; ed un insieme $S \subset V$ di luoghi “sicuri”, dove la popolazione dei villaggi si può rifugiare in caso di eruzione. Notate che $P \cap S = \emptyset$.

Il vostro compito è scrivere un algoritmo per determinare se esiste un insieme di “cammini di evacuazione”, tali per cui

- ogni cammino inizia da un nodo in P e termina in un nodo in S ;
- ogni nodo v in P ha esattamente un cammino che parte da v ;
- i cammini non condividono nessuno degli archi, ovvero tutti gli archi sono attraversati al massimo una volta da un cammino.

Discutere la complessità computazionale del vostro algoritmo.

Soluzione Il problema può essere visto come un problema di flusso. Si crea una supersorgente SS , connessa a tutti i nodi nell'insieme P con archi con peso 1. Si assegna peso 1 a tutti i nodi del grafo originario. Si crea un superpozzo SP connesso a tutti i nodi nell'insieme S con archi con peso $|P|$ (perchè non abbiamo messo alcuna restrizione al numero di cammini che terminano in un nodo in S). Esiste un insieme di cammini di evacuazione se e solo se il flusso massimo ottenuto in questa rete è pari a $|P|$. La complessità è pari a $O(E|f * |)$, dove $|f * | = O(|P|) = O(V)$. Il costo totale è quindi $O(VE)$.

Esercizio 3

Sia G un grafo non orientato, connesso con pesi sugli archi tutti distinti. Dimostrare che esiste un unico minimum spanning tree per G .
Nota: l'esercizio non è difficile, ma va motivata bene la risposta.

Soluzione Si consideri uno spanning tree di costo minimo $T \subseteq E$ ottenuto dall'algoritmo di Kruskal (gli archi vengono selezionati in ordine crescente di peso, evitando di costruire cicli). Per assurdo, si consideri uno spanning tree $T' \subseteq E$ diverso da T che abbia lo stesso peso minimo di T .

Siano $e_1, e_2, \dots, e_n$ gli archi di E ordinati per peso crescente. Si identifichi il più basso valore di j tale per cui $e_j \in T' - T$. Al momento in cui Kruskal ha valutato e_j , l'arco non è stato inserito perchè si veniva a formare un ciclo. Si consideri un'arco e_k di questo ciclo che non faccia parte di T' (tale arco deve esistere, altrimenti T' non sarebbe un albero); inoltre, poiché e_k era già presente nell'albero parziale quando e_j è stato considerato, allora $w(e_k) < w(e_j)$. Quindi l'albero $T' - \{e_j\} \cup \{e_k\}$ è un albero di costo inferiore a T' , il che è assurdo.

Esercizio 4

Per la soluzione di un problema, vi è stato proposto di scegliere fra un algoritmo iterativo che ha complessità $O(n^2)$ e un algoritmo ricorsivo che ha la seguente complessità:

$$T(n) = \begin{cases} n & \text{se } n \leq 1 \\ aT(n/4) + 1 & \text{se } n > 1 \end{cases}$$

Quale algoritmo preferite? Motivate la risposta.

Soluzione Per $a \leq 16$ si ottiene una complessità pari a $O(n^2)$, quindi è l'algoritmo ricorsivo è preferibile (o uguale) all'algoritmo iterativo. Per valori di a maggiori di 16, l'algoritmo iterativo è preferibile.

Algoritmi e Strutture Dati - Prova d'esame

11/02/08

Esercizio 1

Scrivere un algoritmo che, dati i seguenti input

- un grafo orientato con pesi sugli archi non negativi,
- una coppia di vertici s e t ,
- due interi K e W ,

sia in grado di rispondere se esiste un percorso da s a t con peso totale W e numero di archi K .

Descrivere ad alto livello l'idea dell'algoritmo e fornire lo pseudocodice. Illustrare l'algoritmo su un piccolo esempio.

Suggerimento Si può scegliere di definire $P(v, w, k) = \text{true}$ se c'è un percorso da s a v con peso totale w e numero di archi k (per ogni vertice v e ogni intero w e k con $0 \leq w \leq W$ e $0 \leq k \leq K$).

Soluzione

Definiamo

$P(v, 0, 0) = \text{true}$ per ogni vertice v .

$P(v, w, 0) = \text{false}$ per ogni vertice v e per $w > 0$.

Per $k > 0$, $P(v, w, k) = \text{true}$ se e solo se $P(u, w - \text{weight}(u, v), k - 1) = \text{true}$ per qualche arco (u, v) .

L'algoritmo è quindi il seguente:

Algorithm 1 ESISTECAMMINO(G, s, t, W, K)

```
1: Set P[v, 0, 0] = true per ogni vertice v
2: Set P[v, w, 0] = false per ogni vertice v and w = 1, 2, ..., W
3: for k = 1, 2, ..., K do
4:   for w = 0, 1, 2, ..., W do
5:     Set P[v, w, k] = true se esiste un arco (u,v) tale per cui P[u, w - wt(u, v), k - 1] = true (for each vertex v).
6: Return P[t, W, K]
```

Esercizio 2

Dato un albero con radice T , con insieme dei nodi V , e una funzione peso $p: V \rightarrow \mathbb{R}$, chiamiamo costo di un nodo $v \in V$ il valore

$$c(v) = \begin{cases} p(v) & \text{se } v \text{ è la radice} \\ p(v) + c(\text{padre}(v)) & \text{altrimenti} \end{cases}$$

Definire un algoritmo per il seguente problema.

- Istanza: un albero con radice T , rappresentato mediante liste di adiacenza, e per ogni nodo v il peso $p(v) \in \mathbb{R}$.
- Soluzione: l'insieme delle foglie di T che hanno costo minimo.

Svolgere inoltre l'analisi dell'algoritmo valutando il tempo di calcolo e lo spazio di memoria richiesti in funzione del numero n di nodi dell'albero (si assuma il criterio uniforme).

Soluzione

Il problema può essere risolto con una visita in preordine dell'albero T nella quale, per ogni nodo v diverso dalla radice, il valore $c(v)$ viene calcolato conoscendo il costo del padre di v . Descriviamo quindi un programma principale che richiama una procedura ricorsiva, chiamata $Visita()$, che calcola i valori $c(v)$ di ogni nodo v diverso dalla radice. Tale procedura calcola anche una lista F contenente le foglie di T accompagnate dal relativo costo e determina il valore minimo m di tali costi. Il programma principale poi scorrerà la lista F e determinerà l'elenco delle foglie che hanno costo m . Nella sua computazione la chiamata $Visita(v)$ mantiene, oltre alle variabili globali F e m , anche la variabile globale C che rappresenta il costo $c(v)$ del nodo corrente v . Nella descrizione che segue supponiamo che V sia l'insieme dei nodi di T , r sia la radice di T e, per ogni nodo $v \in V$, $L(v)$ sia la lista dei figli di v .

Algorithm 2 MAIN

```
1:  $m = \infty$ 
2:  $F = \text{lista vuota}$ 
3:  $C = p(r)$ 
4:  $\text{Visita}(r)$ 
5:  $S = \text{lista vuota}$ 
6: for  $(v, x) \in F$  do
7:   if  $x = m$  then
8:      $S = \text{InserisciInTesta}(S, v)$ 
9: Return  $S$ 
```

Algorithm 3 VISITA(v)

```
1: if  $L(v)$  is empty (ovvero  $v$  e' una foglia) then
2:    $F = \text{InserisciInTesta}(F, (v, C))$ 
3:   if  $C < m$  then
4:      $m = C$ 
5:   else
6:     for  $w \in L(v)$  do
7:        $C := C + p(w)$ 
8:        $\text{Visita}(w)$ 
9:    $C := C - p(v)$ 
```

Complessita'. La procedura *Visita* viene chiamata esattamente una volta su ogni nodo dell'albero e, per ogni nodo, esegue un numero di operazioni limitato da una costante. Quindi il tempo richiesto dalla visita in preordine dell'albero e' $O(n)$. Le altre operazioni eseguite dal Main richiedono un tempo $O(n)$. Quindi il tempo complessivo richiesto dall'algoritmo e' $O(n)$. Per quanto riguarda lo spazio di memoria l'algoritmo richiede uno spazio $O(n)$ per mantenere l'input (cioe' l'albero T). L'esecuzione della chiamata *Visita*(r) richiede uno spazio $O(n)$ per mantenere la pila che implementa la ricorsione. Una analoga quantita' richiesta per mantenere le liste F e S . Quindi anche lo spazio di memoria richiesto e' dell'ordine $O(n)$.

Esercizio 3

Si consideri un array di n interi. L'array contiene k elementi "fuori ordine", ovvero elementi tali per cui, se fossero rimossi dall'array, i rimanenti $(n - k)$ elementi sarebbero ordinati in ordine ascendente. Si disegni un algoritmo per ordinare un array di questo tipo, piu' efficiente degli algoritmi di ordinamento generici che non possono sfruttare questa informazione addizionale. Si discuta la complessita' dell'algoritmo proposto.

Soluzione

Un contesto reale in cui un algoritmo del genere potrebbe essere richiesto e' il seguente.

Il ministro dell'Interno dello stato Cindia vi chiama e vi dice che mantenevano la lista (anagrafe) dei 100 milioni di cittadini dello stato ordinata ma purtroppo c'e' stato un piccolo problema con la lista e un piccolo numero di cittadini sono stati inseriti non in ordine. Purtroppo Cindia dispone di computer molto molto lenti e quindi la richiesta e' quella di creare un algoritmo piu' efficiente di $O(n \log(n))$ ma qualcosa che sia vicino a $O(n)$.

Arriviamo alla soluzione. Al fine di ordinare questa lista quasi ordinata, possiamo procedere in questa maniera: dapprima attraversiamo la lista e rimuoviamo i k elementi non ordinati (insieme ad altri k elementi) dalla lista in tempo $O(n)$. A questo punto abbiamo due list: la Lista 1, che e' ordinata per definizione, e la Lista 2, che contiene $2k$ elementi non ordinati. Utilizziamo Mergesort per ordinare la Lista 2 in tempo $O(2k * \log(2k)) = O(k * \log(k))$. Dopo si chiama la Merge del Mergesort sulle due lista (ora ordinate) e questo richiede tempo $O(n)$. La complessita' in tempo totale e' quindi $O(n) + O(k \log(k)) + O(n) = O(n + k \log(k))$.

Pseudocodice (vedi prossimo algoritmo)

Prove di correttezza Abbiamo gia' visto che il Mergesort correttamente ordina una lista. Abbiamo gia' anche visto che la Merge del Mergesort correttamente unisce (fa il merge) di due liste ordinate in una singola lista ordinata. Quindi se riusciamo a provare che L e' una lista ordinata quando la Merge e' chiamata, l'array finale deve anche essere ordinato.

Attraversiamo L un elemento alla volta, e verifichiamo ad ogni passo che l'elemento precedente e quello corrente siano in ordine. Se non lo sono, almeno uno dei due elementi deve appartenere all'insieme dei k elementi non ordinati. Li rimuoviamo entrambi dalla lista.

Algorithm 4 SORTKOUTOFORDER(L)

```
1: L2 = lista vuota
2: n = L.length
3: i = 1
4: while i < n do
5:   if L[i] < L[i-1] then
6:     addElement(L2, L[i-1])
7:     addElement(L2, L[i])
8:     removeElementAt(L, i-1)
9:     removeElementAt(L, i)
10:    n = n - 2
11:    if i > 1 then
12:      i = i - 1
13:    else
14:      i = i + 1
15: MergeSort(L2)
16: Merge(L, L2)
```

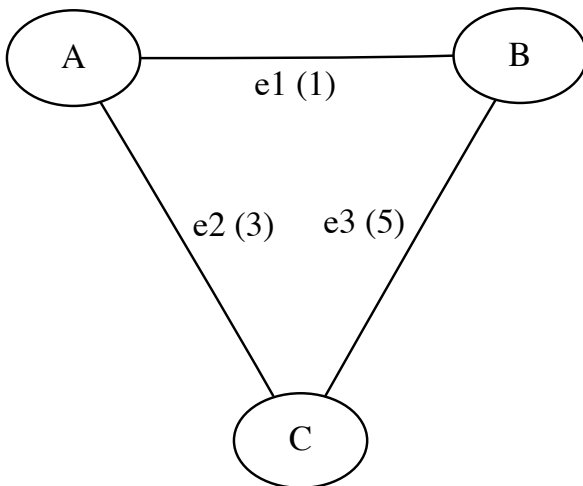
Dopo il primo passo, l'invariante e' ricorsivamente vera per la nuova lista che contiene $n - 2$ elementi, di cui al massimo $k - 1$ sono fuori ordine. Continuiamo attraversando in questa maniera l'array fino a che la fine e' raggiunta e $2k$ elementi sono stati rimossi. Un caso speciale avviene se il primo e il secondo elemento dell'array sono fuori ordine e il caso e' coperto nello pseudocodice.

Esercizio 4

Sia dato un grafo non orientato $G = (V, E)$ e una funzione di pesi $w : E \rightarrow \mathbb{R}$. Siano e_1 l'arco di peso minimo, e_2 il secondo arco di peso minimo e e_3 il terzo arco di peso minimo. Confutare o provare la seguente affermazione: gli archi e_1 , e_2 ed e_3 fanno parte dello stesso albero di copertura minimo.

Soluzione

L'affermazione e' falsa. Basta fornire un controesempio. Il caso piu' semplice e' quando e_1 , e_2 ed e_3 formano un ciclo. Ad esempio il seguente grafo con tre node e tre archi (e_1 , e_2 ed e_3) il cui albero di copertura minimo non contiene e_3 .



Algoritmi e Strutture Dati - Prova d'esame

30/06/08

Esercizio 1

Sia $A[n, n]$ una matrice quadrata di interi distinti. Si supponga che le righe di A siano ordinate in modo crescente da sinistra verso destra. Si assuma, inoltre, che le colonne di A siano ordinate in modo crescente dal basso verso l'alto. Si consideri il problema di determinare se, dato un intero k , k compare in A .

- Si definisca un algoritmo di complessità strettamente inferiore ad $O(n^2)$ per il problema proposto;
- si valuti la correttezza e la complessità dell'algoritmo definito al passo precedente.

Soluzione Il problema può essere risolto in vari modi; ovviamente, una ricerca esaustiva comporta un costo di $O(n^2)$ nel caso pessimo. Una possibilità è la seguente: si conseri l'elemento $A[n, n]$ della matrice A .

- Se $A[n, n] = k$, abbiamo terminato.
- Se $A[n, n] < k$, allora tutti gli elementi della matrice nella n -esima riga non possono contenere k , perché sono tutti minori di $A[n, n]$.
- Se $A[n, n] > k$, allora tutti gli elementi della matrice nella n -esima colonna non possono contenere k , perché sono tutti maggiori di $A[n, n]$.

Si prosegue considerando la matrice A senza la riga o la colonna corrispondente.

chiamata alla funzione `search_element(A, k, n, n)`

```
search_element(A, k, i, j)
  if (i < 1 or j < 1) then
    return false
  if A[i, j] == k then
    return true
  else if A[i, j] < k then
    return search_element(A, k, i-1, j)
  else
    return search_element(A, k, i, j-1)
```

Esercizio 2

Sia $G = (V, E)$ un grafo orientato e si assuma che i nodi in G siano colorati mediante il colore rosso oppure verde. Si consideri il problema di determinare se G possiede un ciclo semplice contenente solo nodi verdi.

- Si proponga un algoritmo efficiente per il problema proposto;
- Si valuti la correttezza e la complessità della procedura delineata.

Soluzione E' sufficiente fare una visita in ampiezza del grafo, partendo solo e unicamente dai nodi verdi non già visitati, e non visitando mai nodi rossi - il che corrisponde ad eliminare i nodi rossi dal grafo, ed ottenere potenzialmente un grafo non connesso. Il costo totale è $O(V + E)$.

Esercizio 3

Sia $A[1 \dots n]$ un vettore contenente $n = 3m$ interi distinti. Si consideri il problema di stampare gli elementi di A maggiori o uguali ad almeno m interi in A e minori o uguali ad almeno m interi in A .

- Si proponga un algoritmo lineare per risolvere il problema proposto;
- Si discuta la correttezza e la complessità dell'algoritmo definito.

Nota: si consiglia di **utilizzare** un algoritmo noto, senza doverlo riscrivere.

Soluzione Qui l'idea era quello di utilizzare il problema della selezione, che permette di individuare la posizione dell'elemento in posizione m e $2m$ nell'array dei valori ordinati, senza effettivamente ordinarli. Il costo di tali operazioni è $O(n)$. A questo punto, si scorre tutto l'array e si individuano i valori superiori e inferiori, come richiesto.

```
print_one_third(A, n)

m := n/3
min := select(A, 1, n, m)
max := select(A, 1, n, 2*m)
for i := 1 to n do
    if (A[i] > min and a[i] < max) then
        print A[i]
```

dove `select(A, p, r, i)` illustrata nei lucidi.

Esercizio 4

Determinare tra le seguenti relazioni quali sono vere e quali sono false. Dimostrare

1. $O(n) - n + 1 = O(1)$
2. $n^2(2 + \cos n) - n = \Theta(n^2)$
3. $O(n \log n) / \log n = \Theta(n)$

Soluzione

- Falsa; $O(n)$ contiene tutte le funzioni che crescono asintoticamente meno di $f(n) = cn$, per qualche costante c ; quindi sottrarre n non cambia la situazione.
- Vera; $\cos n$ è un termine $O(1)$
- Vera

Cognome:

Nome:

Matricola:

Riga:

Col:

Algoritmi e Strutture Dati - Prova d'esame - Problemi
01/09/09

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna.

Esercizio 1

Per ognuna delle seguenti coppie di funzioni $f(n)$ e $g(n)$, proporre un costante c appropriata tale per cui $f(n) \leq c \cdot g(n)$ per tutti gli $n > 1$.

1. $f(n) = n^2 + n + 1, g(n) = 2n^3$
2. $f(n) = n\sqrt{n} + n^2, g(n) = n^2$
3. $f(n) = n^2 - n + 1, g(n) = n^2/2$

Esercizio 2

Siano dati k vettori ordinati di dimensione n . Scrivere uno o più algoritmi che copino tutti i valori contenuti nei vettori in un unico vettore ordinato di dimensione kn . Discutere la complessità degli algoritmi proposti e confrontarli in base ai fattori k e n .

Esempi di possibili complessità:

1. $T(n) = kn \log(kn)$
2. $T(n) = k^2n$
3. $T(n) = kn \log k$

Esercizio 3

Un “grafo di intervalli” è un grafo non orientato $G = (V, E)$ i cui vertici corrispondono ad intervalli sulla linea reale, dove ogni intervallo è specificato da un valore iniziale v_s ed un valore finale v_t . Due vertici in G sono connessi se e solo se gli intervalli corrispondenti sono sovrapposti. Supponiamo di avere un grafo di intervalli G con associati i rispettivi intervalli. Proporre un algoritmo efficiente per trovare un insieme indipendente massimale in G (si veda appunti su P/NP per definizione di insieme indipendente).

Esercizio 4

Nel “minimum product spanning tree”, il costo di un albero è dato dal prodotto di tutti i pesi dell'albero, invece della somma. Si assuma che tutti i vertici abbiano un peso positivo.

Scrivere un algoritmo che calcoli il minimum product spanning tree (suggerimento: pensate ai logaritmi). Discutere la correttezza e la complessità dell'algoritmo proposto.

Algoritmi e Strutture Dati - Prova d'esame

01/09/09

Esercizio 1

1. La disequazione è vera per qualsiasi costante $c \geq 7/16$; (si può ottenere notando che per $n > 1$, $f(n)$ è sempre maggiore di $g(n)$; quindi basta sostituire $n = 2$ per ottenere il valore minore di c)
2. Simili considerazioni si possono fare per questa disequazione, ottenendo $c \geq \frac{\sqrt{2}+2}{2}$
3. $c \geq 2$

Esercizio 2

Esistono almeno tre soluzioni possibili.

La prima consiste nel copiare tutti i vettori nel vettore di output di dimensione kn e poi applicare un algoritmo di ordinamento. Il costo di questo algoritmo è chiaramente $O(kn \log(kn))$.

Una seconda possibilità è quella di utilizzare l'algoritmo `merge` visto per l'algoritmo mergesort, scegliendo ad ogni passo il minimo fra gli elementi di testa di k vettori. Selezionare il minimo costa $O(k)$, e questa operazione ripetuta kn volte risulta in una complessità di $O(k^2n)$.

Un algoritmo più furbo (ovviamente a seconda della dimensione di k) potrebbe utilizzare uno heap di dimensione k per selezionare il minimo ad ogni passo, per un costo totale di $O(kn \log k)$.

Vediamo l'algoritmo. Utilizziamo uno heap in grado di contenere delle triple $\langle v, i, r \rangle$, dove v è il valore rispetto a cui si ordina, i è l'indice del vettore da cui è stato preso il valore v , r è l'indice del vettore $A[i]$ in cui si trova il valore v . Con queste informazioni, siamo in grado di rimuovere un valore dallo heap ed avanzare nel vettore da cui questo valore è stato preso.

Procedure MERGE(integer[][] A , integer k , integer n)

```
h = newHeap(k)
for i ← 1 to k do
  h.insert(⟨A[i][1], i, 1⟩)
for j ← 1 to kn do
  ⟨v, i, r⟩ ← h.min()
  B[j] ← v
  h.deleteMin()
  if r < n then
    h.insert(⟨A[i][r+1], i, r+1⟩)
```

Si noti che le operazioni `min` e `insert` leggono e scrivono la tripla, di cui come si è detto il primo valore è quello su cui si effettua l'ordinamento.

Esercizio 3

E' possibile utilizzare l'algoritmo greedy per la selezione di attività (insieme massimale di intervalli non sovrapposti); questo corrisponde all'insieme indipendente massimale sul grafo. Infatti, se due nodi non hanno un arco adiacente, i corrispondenti intervalli non sono sovrapposti; se due intervalli non sono sovrapposti, non c'è arco che li collega.

Esercizio 4

E' possibile notare che $\log(a \cdot b) = \log a + \log b$; quindi è sufficiente cambiare tutti i pesi degli archi, sostituendoli il peso $w(u, v)$ con $\log w(u, v)$.

Cognome:

Nome:

Matricola:

Riga:

Col:

Algoritmi e Strutture Dati - Prova d'esame - Problemi
19/04/10

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su entrambi i fogli.

Esercizio 1 - Punti ≥ 4

Ordinare le seguenti funzioni in accordo alla loro complessità asintotica. Si scriva $f(n) < g(n)$ se $\Theta(f(n)) \subset \Theta(g(n))$. Si scriva $f(n) = g(n)$ se $\Theta(f(n)) = \Theta(g(n))$.

Le funzioni da ordinare:

$$\begin{aligned}f_1(n) &= 2^{n+2} \\f_2(n) &= \log^2 n \\f_3(n) &= (\log_n(\sqrt{n})^2 n) + 1/n^2 \\f_4(n) &= 3n^{0.5} \\f_5(n) &= 16^{n/4} \\f_6(n) &= 2\sqrt{n} + 4n^{1/4} + 8n^{1/8} + 16n^{1/16} \\f_7(n) &= \sqrt{(\log n)(\log n)} \\f_8(n) &= n^{\log 4n} \\f_9(n) &= \frac{n^3}{(n+1)(n+3)} \\f_{10}(n) &= 2^n\end{aligned}$$

Esercizio 2 - Punti ≥ 6

Data la seguente equazione di ricorrenza,

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/3) + T(2n/3) + 1 & n > 1 \end{cases}$$

fornire un limite inferiore e un limite superiore alla complessità asintotica della funzione $T(n)$.

Esercizio 3 - Punti ≥ 8

L'azienda Algorithm-R-us è quotata in borsa. Supponete di avere tabulato nel vettore $A[1 \dots n]$ la quotazione in borsa dell'azienda lungo un periodo di tempo lungo n giorni, e che $A[1] < A[n]$. E' possibile dimostrare che esiste almeno una coppia di giorni consecutivi $i, i+1$ tale per cui $A[i] < A[i+1]$. Scrivere un algoritmo $O(\log n)$ che restituisca un indice i in cui questo avviene. Si valuti la complessità.

Esercizio 4 - Punti ≥ 8

Avete a disposizione il grafo delle amicizie di Facebook, realizzato tramite liste di incidenza. Volete calcolare la persona che ha il maggior numero di "amici e amici di amici", ovvero nodi a distanza 1 o 2 da esso. Scrivere un algoritmo che ritorna questo valore. Calcolare la complessità.

Esercizio 5 - Punti ≥ 10

Un albero binario "pieno" è un albero binario in cui tutti i nodi hanno esattamente 0 o 2 figli, e nessun nodo ha 1 figlio. Scrivere un programma ricorsivo che valuti P_n , ovvero il numero di alberi binari strutturalmente diversi che si possono ottenere con n nodi. Si valuti la complessità dell'algoritmo risultante.

Algoritmi e Strutture Dati - Prova d'esame

19/04/2010

Esercizio 1

Le funzioni da ordinare:

$$\begin{aligned}f_1(n) &= 2^{n+2} = 4 \cdot 2^n = \Theta(2^n) \\f_2(n) &= \log^2 n = \Theta(\log^2 n) \\f_3(n) &= (\log_n(\sqrt{n})^2 n) + 1/n^2 = (\log_n n^2) + 1/n^2 = 2 + 1/n^2 = \Theta(1) \\f_4(n) &= 3n^{0.5} = \Theta(n^{1/2}) \\f_5(n) &= 16^{n/4} = (2^4)^{n/4} = 2^{4n/4} = \Theta(2^n) \\f_6(n) &= 2\sqrt{n} + 4n^{1/4} + 8n^{1/8} + 16n^{1/16} = \Theta(n^{1/2}) \\f_7(n) &= \sqrt{(\log n)(\log n)} = \Theta(\log n) \\f_8(n) &= n^{\log 4n} = n^{\log 4 + \log n} = n^2 + n^{\log n} = \Theta(n^{\log n}) \\f_9(n) &= \frac{n^3}{(n+1)(n+3)} = \Theta(n) \\f_{10}(n) &= 2^n = \Theta(2^n)\end{aligned}$$

Una volta stabilito l'ordine Θ delle funzioni, è abbastanza semplice stabilire l'ordine corretto:

$$f_3 < f_7 < f_2 < f_4 = f_6 < f_9 < f_8 < f_1 = f_5 = f_{10}$$

L'unica eccezione è la posizione della funzione f_8 . Da un lato, è facile vedere che dato un qualunque polinomio di grado k , esiste un indice n_0 tale per cui $\log n > k$, per ogni valore $n > n_0$. In altre parole, qualunque polinomio si scelga, per valori sufficientemente grandi di n la funzione f_8 crescerà più velocemente di n^k .

Confrontando la funzione $n^{\log n}$ e 2^n , è possibile applicare la funzione logaritmo ad entrambi, ottenendo $\log n \cdot \log n$ e n , rispettivamente. Poichè $\Theta(\log f_8(n)) \subset \Theta(\log 2^n)$, se ne deduce che $\Theta(f_8(n)) \subset \Theta(2^n)$.

Esercizio 2

È facile osservare le seguenti relazioni:

$$T(n) = 2T(n/3) + T(2n/3) + 1 \geq 2T(n/3) + T(n/3) = 3T(n/3) + 1$$

e

$$T(n) = 2T(n/3) + T(2n/3) + 1 \leq 2T(2n/3) + T(2n/3) = 3T(2n/3) + 1$$

Applicando il master theorem alle due relazioni, si ottiene che:

$$T(n) \geq 3T(n/3) + 1 = \Theta(n)$$

e

$$T(n) \leq 3T(2n/3) + 1 = \Theta(n^{\log_{3/2} 3}) = T(n^{2.70\dots})$$

È possibile dimostrare un limite più stretto provando a dimostrare che $T(n) = O(n^2)$ per sostituzione. Nel caso generale, dobbiamo dimostrare che esistono $c > 0$, $m \geq 0$ tale per cui $T(n) \leq cn^2$ per ogni $n \geq m$.

$$\begin{aligned}T(n) &= 2T(n/3) + T(2n/3) + 1 \\&\leq 2cn^2/9 + 4cn^2/9 + 1 \\&= 2/3cn^2 + 1 \leq cn^2.\end{aligned}$$

L'ultima condizione è vera per $n \geq m = 1$, $c \geq 3$.

Nel caso base, $T(1) = 1 \leq c \cdot 1^2$, che è vera per $c \geq 1$.

Esercizio 3

La soluzione proposta è basata sulla tecnica divide-et-impera. Generalizzando, vogliamo risolvere il problema di identificare due valori consecutivi crescenti in un array $A[i \dots j]$, con $A[i] < A[j]$. Il problema originale corrisponde a $A[1 \dots n]$.

$A[i \dots j]$ può essere ridotto ad **uno** dei sottoproblemi $A[i \dots m]$ e $A[m \dots j]$, dove $m = \lfloor (i+j)/2 \rfloor$ è l'indice mediano fra i e j , in base alle seguenti osservazioni:

- Se $A[i] < A[m]$, allora è possibile considerare il solo sottoproblema $A[i \dots m]$, in cui il primo estremo è minore dell'ultimo.
- Se $A[m] \leq A[i] < A[j]$, allora è possibile considerare il solo sottoproblema $A[m \dots j]$, in cui il primo estremo è minore dell'ultimo.

Il caso base avviene quando siamo rimasti con due soli elementi.

Si noti che applicare ricorsivamente la funzione ad entrambi i sottovettori è inutile e porta ad una complessità di $O(n)$; si noti che è possibile scrivere facilmente questo algoritmo in maniera iterativa.

```
integer trova(ITEM [ ] A, integer i, integer j)
```

```
  if i + 1 = j then
    | return i
  else
    | integer m ← ⌊(i + j)/2⌋
    | if A[i] < A[m] then
    |   | return trova(A, i, m)
    | else
    |   | return trova(A, m, j)
```

Chiamata $\text{trova}(A, 1, n)$.

Esercizio 4

In questo esercizio non era spiegato chiaramente come calcolare il numero di amici e amici di amici. Erano possibili tre soluzioni diverse:

1. tre cicli annidati su tutti i nodi; questo approccio, oltre a non tener conto dei duplicati, è inefficiente $-O(n^3)$
2. utilizzando “programmazione dinamica”, è possibile salvare in ogni nodo il numero di amici e ottengo lo stesso risultato di cui sopra, ma in $O(n + m)$ (che è al limite $O(n^2)$).
3. volendo evitare i duplicati, possiamo fare n visite BFS, in cui però limitiamo a due il numero massimo di passi; l'algoritmo resta comunque $O(n^3)$, ma abbiamo evitato i duplicati.

Mostriamo qui la terza soluzione.

integer facebook3(**GRAPH** G)

```
integer  $max \leftarrow 0$ 
foreach  $v \in G.V()$  do
  integer  $t \leftarrow \text{conta}(\text{GRAPH } G, \text{NODE } v)$ 
  if  $t > max$  then
     $max \leftarrow t$ 
return  $max$ 
```

integer conta(**GRAPH** G , **NODE** r)

```
boolean[]  $visitato \leftarrow \text{new boolean}[1 \dots G.n]$ 
foreach  $v \in G.V() - \{r\}$  do  $visitato[v] \leftarrow \text{false}$ 
 $visitato[r] \leftarrow \text{true}$ 
integer  $f \leftarrow 0$ 
foreach  $u \in G.\text{adj}(r)$  do
   $visitato[u] \leftarrow \text{true}$ 
   $f \leftarrow f + 1$ 
  foreach  $v \in G.\text{adj}(u)$  do
    if not  $visitato[v]$  then
       $visitato[v] \leftarrow \text{true}$ 
       $f \leftarrow f + 1$ 
return  $f$ 
```

Esercizio 5

Il numero di alberi pieni strutturalmente diversi è calcolato nel modo seguente. Innanzitutto, è impossibile costruire un albero pieno se n è pari. Questo perché gli alberi pieni si ottengono partendo dalla radice e aggiungendo via via coppie di figli. Per i valori dispari, il numero di nodi pieni è ottenuto ricorsivamente partendo dal caso base $n = 1$ (che corrisponde ad un albero solo). Per $n > 1$, si crea una radice e poi si dividono i rimanenti $n - 1$ figli fra il sottoalbero destro e il sottoalbero sinistro. Se i nodi vanno a destra, $1 \leq i \leq n - 2$, $(n - 1) - i$ nodi vanno a sinistra.

$$T[n] = \begin{cases} 0 & \text{se } n \text{ è pari} \\ 1 & \text{se } n = 1 \\ \sum_{i=1}^{n-2} T(i) \cdot T((n-1) - i) & \end{cases}$$

È possibile calcolare il valore con un semplice programma ricorsivo, ma questo avrebbe complessità superpolinomiale. Per velocizzare l'algoritmo, è opportuno utilizzare la programmazione dinamica.

integer alberipieni(**integer** n)

```
if  $n$  è pari then
  return 0
else
  integer[]  $D \leftarrow \text{new integer}[1 \dots n]$ 
   $D[1] \leftarrow 1$ 
  for integer  $i = 3$  to  $n$  step 2 do
     $D[i] \leftarrow 0$ 
    for integer  $j = 1$  to  $i - 2$  step 2 do
       $D[i] \leftarrow D[i] + D[j] \cdot D[(n-1) - j]$ 
return  $D[n]$ 
```

La complessità di questo algoritmo è $O(n^2)$.

Cognome:

Nome:

Matricola:

Riga:

Col:

Algoritmi e Strutture Dati - Prova d'esame - Problemi
9/06/10

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su entrambi i fogli.

Esercizio 1 - Punti ≥ 8

Dato un vettore P di n interi in cui $P[i]$ è il prezzo di una certa merce al giorno i , in cui conviene comprare quella merce e il giorno j , con $j > i$, in cui conviene rivenderla in modo da massimizzare il profitto o in alternativa minimizzare le perdite. In altre parole siamo interessati a conoscere la coppia (i, j) con $i < j$ per cui risulta massimo il valore $P[j] - P[i]$. Calcolare la complessità e dimostrare la correttezza. È possibile risolvere il problema in $O(n)$.

Esercizio 2 - Punti $\geq 6-2-6$

State viaggiando con una modernissima auto elettrica su un'autostrada lunga N km; prima che la vostra batteria si esaurisca (dopo r km), dovete fermarvi in un'area di servizio e cambiarla con una nuova batteria. Siano $D[1 \dots n]$ e $C[1 \dots n]$ due array di interi, dove $D[i]$ è la distanza dell'area di servizio i -esima dall'inizio dell'autostrada, e $C[i]$ è il costo di una nuova batteria nell'area i .

1. Descrivete un algoritmo greedy che minimizzi il numero di soste;
2. Dimostrate con un controesempio che il vostro algoritmo non funziona se volete invece minimizzare il costo totale del viaggio;
3. Descrivete un algoritmo basato su programmazione dinamica che minimizzi il costo totale del viaggio.

Nei casi (1) e (3), dimostrate la correttezza del vostro algoritmo e calcolatene la complessità.

Esercizio 3 - Punti ≥ 8

Un grave terremoto ha colpito una regione. Ci sono n persone ferite dislocate nell'area colpita, e k ospedali sono ancora funzionanti. Ogni persona deve essere portata in un ospedale che non disti più di mezz'ora dal luogo di ferimento. Si vuole evitare di distribuire i pazienti in maniera diseguale, così ogni ospedale deve ricevere al più $\lceil n/k \rceil$ pazienti. Scrivere un algoritmo che lavori in tempo polinomiale che distribuisca i pazienti fra gli ospedali, e calcolarne la complessità.

Algoritmi e Strutture Dati - Prova d'esame

09/06/2010

Esercizio 1

Mi calcolo, per tutti gli indici consecutivi, la differenza (positiva o negativa): $H[i] = P[i + 1] - P[i]$. Questo può essere fatto in tempo $O(n)$. Applico poi l'algoritmo $O(n)$ che abbiamo visto per ottenere la sottosequenza consecutiva di somma massima.

Esercizio 2

Esercizio 2.1 Per minimizzare il numero di fermate, è sufficiente fermarsi ogni volta (in maniera greedy) all'ultima stazione di servizio possibile, prima di rimanere a secco. Per semplificare l'esercizio, supponiamo che esista un'ultima fermata $D[n + 1] = N$ con costo $C[n + 1] = 0$, come sentinella. Inoltre, assumiamo che sia possibile completare il percorso con un'autonomia di r km.

SET fermate(integer[] D , integer n)

$deadline \leftarrow 0$

SET stops $\leftarrow$ Set()

for $i \leftarrow 1$ to n do

 if $D[i] \leq deadline$ and $D[i + 1] > deadline$ then

 stops.insert(i)

$deadline \leftarrow D[i] + r$

return stops

Il costo di questo algoritmo è chiaramente $O(n)$. La sottostruttura ottima è dimostrabile semplicemente facendo notare che se viene effettuata una ricarica alla stazione i , ci si riduce al problema $D[i \dots n]$ con una lunghezza della strada pari a $N - D[i]$; qualunque soluzione ottima per questo problema fa parte della soluzione ottima del problema originale.

La scelta greedy può essere dimostrata nel modo seguente: assumiamo che esista una soluzione S che non includa la stazione k , ultima possibile a partire dall'inizio. Sia i la prima stazione utilizzata in S , con $i < k$; se sostituisco i con k , ottengo una soluzione $S' = S - \{i\} \cup \{k\}$ che è comunque una soluzione ottima (ha la stessa dimensione) e rispetta tutti i vincoli (k è raggiungibile dall'inizio e può raggiungere la stazione successiva in S , perchè questa era raggiungibile da i che era più indietro).

Esercizio 2.2 Basta scegliere $N = 3$, $r = 2$, $n = 2$, $D[1] = 1$, $D[2] = 2$, $C[1] = 1$, $C[2] = 2$. In altre parole, per percorrere l'autostrada lunga 3km con autonomia 2, bisogna fare una sosta. Il nostro algoritmo sceglie la seconda stazione, che però ha un costo superiore.

Esercizio 2.3 La sottostruttura ottima è stata dimostrata al punto precedente. E' possibile definire ricorsivamente il problema come segue. Aggiungiamo le stazioni fittizie $D[0]$ e $D[n + 1]$, al km 0 e N , con costo $C[0] = C[n + 1] = 0$.

Il problema $P(i)$ consiste nell'individuare il costo minimo $M[i]$ da pagare nel caso si acquisti la batteria alla stazione i e si debba raggiungere la stazione $n + 1$. Il problema iniziale è quindi $P(0)$.

$M[i]$ per risolvere il problema $P(i)$ può essere definito ricorsivamente come segue:

$$M[i] = \begin{cases} 0 & \text{se } i = n + 1 \\ \min_{j: j > i \wedge D[j] \leq D[i] + r} \{M[j]\} + C[i] & \text{altrimenti} \end{cases}$$

Inviduare l'indice j per cui $D[j] \leq D[i] + r$ può richiedere $O(n)$; quindi il costo pessimo di tale algoritmo è $O(n^2)$.

Notate che sarebbe possibile realizzare un algoritmo di costo $O(nr)$ creando una tabella bidimensionale $n \times r$. L'algoritmo risultante sarebbe psuedopolinomiale.

Il codice è il seguente:

```

mincosto(integer[] D, integer[] C, integer n)
integer[] M ← new integer[0 ... n + 1]
M[n + 1] ← 0
for i ← n downto 0 do
    integer j ← i + 1
    integer M[i] = +∞
    while j ≤ n + 1 and D[j] ≤ D[i] + r do
        M[i] ← min(M[i], M[j])
        j ← j + 1
    M[i] ← M[i] + C[i]
return M[0]

```

Esercizio 3

È possibile trasformare questo problema in un una rete di flusso. Si aggiunge un nodo $p_1 \dots p_n$ per ognuna delle persone; un nodo $o_1 \dots o_k$ per ognuno degli ospedali; un nodo sorgente e un nodo pozzo. Ogni persona è unita alla sorgente con un arco di peso 1, ad indicare che ogni persona dovrebbe essere collocata al massimo in un ospedale. Ogni persona è collegata agli ospedali che può raggiungere in meno di mezz'ora con un arco di peso 1. Ogni ospedale è collegato al pozzo con un arco di peso $\lceil n/k \rceil$, ad indicare che ogni ospedale non dovrebbe accogliere un numero maggiore di pazienti.

Il flusso massimo non può superare n ; se ha valore n , ogni paziente viene assegnato ad un ospedale. Il costo per l'esecuzione dell'algoritmo di Ford-Fulkerson è $O(|f^*|(|V| + |E|))$; sapendo che $|f^*| = O(n)$, $V = n + k + 2$, $E = O(kn + n + k)$, si ottiene un costo pari a $O(n(n + k + 2 + kn + k + n)) = O(n^2k)$.

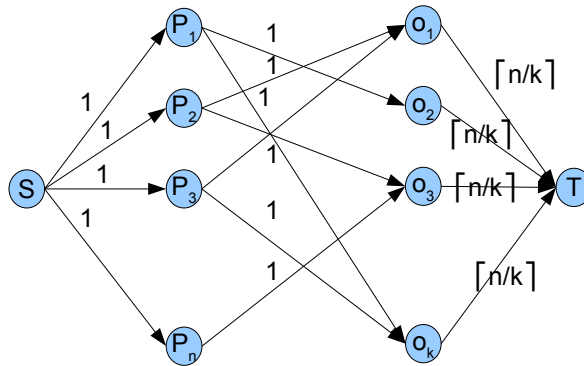


Figura 1: La rete di flusso definita nell'esercizio

Cognome:

Nome:

Matricola:

Riga:

Col:

Algoritmi e Strutture Dati - Prova d'esame - Problemi
22/06/10

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su entrambi i fogli.

Esercizio 1 - Punti ≥ 6

Data una stringa composta da 0 e 1, trovare la lunghezza della più lunga sottostringa contenente un numero k di 0 consecutivi seguita da un numero k di 1 consecutivi. Calcolare la complessità e spiegarne il funzionamento.

Si noti che è possibile che immediatamente prima della vostra sottostringa vi siano ancora 0 o che immediatamente dopo la vostra sottostringa vi siano ancora 1, ma non può accadere contemporaneamente, altrimenti la vostra sottostringa non è la più lunga.

Esercizio 2 - Punti $\geq 10+3$

Siamo nel 2020. Le ferrovie tedesche-austriache (DB, OBB), dopo aver conquistato la tratta del Brennero, hanno fatto fallire Trenitalia e dominano il mercato italiano. La loro politica dei prezzi è la seguente: è possibile acquistare un abbonamento per X euro con validità un mese a partire dal giorno di emissione, valido su tutte le tratte. Oppure pagare un biglietto a tariffa intera per un particolare viaggio. Siete un rappresentante di commercio e avete programmato una serie di viaggi per il prossimo anno, nei giorni $d_1, \dots, d_n$. La tariffa intera per ognuno di questi viaggi è $f_1, \dots, f_n$. Scrivete un algoritmo che minimizzi il costo totale, acquistando un abbonamento nei momenti opportuni.

Punteggio aggiuntivo se il vostro algoritmo, oltre a ritornare il costo minimo, stampa anche i giorni in cui comprare un abbonamento. Valutare la complessità e discutere la correttezza del vostro algoritmo.

Esercizio 3 - Punti ≥ 7

Trovare un limite asintotico superiore e un limite asintotico inferiore alla seguente ricorrenza, facendo uso del metodo di sostituzione:

$$T(n) = 2T(n/8) + 2T(n/4) + n \log n$$

Esercizio 4 - Punti ≥ 4

Una semplice domanda. Supponete di avere un min-heap di n elementi, e di stare cercando il valore **massimo** (non minimo). In quali posizioni del vettore cercate? Giustificare la risposta.

Algoritmi e Strutture Dati - Prova d'esame

22/06/2010

Esercizio 1

Un algoritmo semplice scorre la stringa cercando tutte le sequenze di 0 e 1 consecutivi, le misura e mantiene una variabile *max* per mantenere le informazioni sulla sequenza più lunga.

```
zerouno(integer[] S, integer n)
integer conta0 ← iff(S[1] = 0, 1, 0)
integer conta1 ← iff(S[1] = 1, 1, 0)
integer max ← 0
for integer i ← 2 to n do
    if S[i] = 1 then
        if S[i - 1] = 0 then conta1 ← 0
        conta1 ← conta1 + 1
    else
        if S[i - 1] = 1 then
            max = max(max, min(conta0, conta1))
            conta0 ← 0
        conta0 ← conta0 + 1
return max
```

Esercizio 2

Innanzitutto, assumiamo che i giorni siano rappresentati da un intero fra 1 e 365 e un abbonamento duri 30 giorni (dal giorno di inizio a giorno di inizio+29 inclusi).

Una scelta greedy è la seguente: per ogni giorno x , valuta se il costo di un abbonamento a partire da x è inferiore alla somma delle tariffe nel periodo $[x, x + 29]$; se questo accade, acquista l'abbonamento. Questo non funziona, ad esempio con 31 viaggi in giorni consecutivi, di cui il primo costa meno dell'ultimo.

Si utilizza invece la programmazione dinamica. Sia $C[i]$ il costo minimo per effettuare i viaggi $[i \dots n]$, senza che sia attivo un abbonamento al giorno $d[i]$. Il problema iniziale è calcolare $C[1]$.

Al giorno i -esimo, è possibile (i) acquistare un abbonamento o (ii) pagare la tariffa del giorno. Nel primo caso, si cerca il primo giorno j dopo la scadenza dell'abbonamento; nel secondo caso, si guarda il giorno $i + 1$. Si prende quindi il minimo fra $f[i] + C[i + 1]$ e $X + C[j]$.

$$C(i) = \begin{cases} 0 & i > n \\ \min\{f[i] + C[i + 1], X + C[\min\{j : d[j] \geq d[i] + 30\}]\} & \text{altrimenti} \end{cases}$$

assumendo che $d[n + 1] = +\infty$ che agisce come sentinella.

```
ferrovia(integer[] d, integer[] f, integer n)
integer[] C ← new integer[1 .. n + 1]
boolean[] B ← new boolean[1 .. n]
C[n + 1] ← 0
for i ← n downto 1 do
    j ← i
    while j ≤ n + 1 and d[j] < d[i] + 30 do
        j ← j + 1
    B[i] ← (f[i] + C[i + 1] > X + C[j])
    C[i] ← iff(B[i], X + C[j], f[i] + C[i + 1])
for i ← 1 to n do
    if B[i] then
        print Abbonamento d[i]
print Costo totale: C[1]
```

Il vettore C mantiene il costo per il sottoproblema $[i \dots n]$, mentre $B[i]$ contiene **true** se e solo bisogna comprare l'abbonamento nel giorno i -esimo.

Esercizio 3

Il limite inferiore è banalmente $\Omega(n \log n)$:

$$T(n) = 2T(n/8) + 2T(n/4) + n \log n \geq n \log n = \Omega(n \log n)$$

Il limite superiore è ancora $n \log n$. Lo dimostriamo per sostituzione; supponiamo quindi che esistano c, n_0 tali che $T(n) \leq cn \log n$ per ogni $n \geq n_0$.

$$\begin{aligned} T(n) &= 2T(n/8) + 2T(n/4) + n \\ &\leq 2cn/8 \log n/8 + 2cn/4 \log n/4 + n \log n \\ &\leq cn/4 \log n + cn/2 \log n + n \log n \\ &= 3/4cn \log n + n \log n \leq cn \log n \end{aligned}$$

L'ultima disequazione è vera per $c \geq 4$.

Il caso base non è dimostrabile per $n = 1$, ma è facile vedere che è rispettato per $n \leq 16$.

Esercizio 4

Ogni posizione interna (non-foglia) i ha almeno un figlio $2i$ che ha valore maggiore di quello in i . Quindi non può essere massimo. Si può quindi cercare nelle sole posizioni $[\lfloor n/2 \rfloor + 1 \dots n]$, il cui numero è comunque $O(n)$.

Algoritmi e Strutture Dati - Prova d'esame - Problemi
19/07/10

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna su entrambi i fogli.

Esercizio 1 - Punti $\geq 12+2$

Dato un insieme di m stringhe dette *primitive* ed una stringa $X = x_1 \dots x_n$, si vuole determinare se X è ottenibile dalla concatenazione di stringhe primitive. Ad esempio: dato l'insieme di primitive $\{01, 10, 011, 101\}$, per la stringa $X = 0111010101$ la risposta è sì (due possibili soluzioni sono 011-10-10-101 e 011-101-01-01) mentre per la stringa $X = 0110001$ la risposta è no.

- Descrivere in pseudo-codice un algoritmo che risolve il problema. Spiegare perché l'algoritmo proposto è corretto. (Suggerimento: programmazione dinamica).
- Modificare l'algoritmo proposto in modo che, nel caso X sia ottenibile dalla concatenazione di primitive, vengano prodotta in output la stringa originale in cui la separazione fra stringhe primitive sia evidenziata da "-". Ad esempio: data la stringa $X = 0111010101$ e le primitive $Y_1 = 01$, $Y_2 = 10$, $Y_3 = 011$ e $Y_4 = 101$, l'algoritmo deve produrre in output la sequenza 011-10-10-101 o la sequenza 011-101-01-01.

Esercizio 2 - Punti ≥ 4

Si supponga di rappresentare mediante una coppia di interi $[l, u]$ con $l \leq u$, l'intervallo di interi compresi tra l ed u (estremi inclusi). Sia dato l'insieme di n intervalli $S = \{[l_1, u_1], [l_2, u_2], \dots, [l_n, u_n]\}$. Si descriva lo pseudocodice di un algoritmo efficiente per determinare se esistono (almeno) due intervalli disgiunti in S , se ne discuta la correttezza e la complessità.

Esercizio 3 - Punti ≥ 7

Trovare un limite superiore ed inferiore per la funzione:

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n-1) + \log n & \text{altrimenti} \end{cases}$$

Esercizio 4 - Punti ≥ 7

La larghezza di un albero ordinato è il numero massimo di nodi che si trovano tutti al medesimo livello. Si fornisca una funzione che calcoli in tempo ottimo la larghezza di un albero ordinato di n nodi.

Algoritmi e Strutture Dati - Prova d'esame

19/07/2010

Esercizio 1

Sia $T[i]$ pari a vero o falso a seconda che la stringa $X(i)$ (prefisso i -esimo di X) sia generabile dalle primitive o meno. È facile notare che $T[0] = \mathbf{true}$ (la stringa vuota è generabile da qualunque insieme di primitive), mentre la $T[i] = \mathbf{true}$ se e solo se esiste un indice $j < i$ tale che $T[j] = \mathbf{true}$ e la sottostringa $x_{j+1}x_{j+2} \dots x_i = p[k]$, dove $p[k]$ è una delle stringhe primitive la cui lunghezza è $|p[k]|$. Formalmente:

$$T[i] = \begin{cases} \mathbf{true} & i = 0 \\ \exists k \in \{1, 2, \dots, m\} : T[i - |p[k]|] \wedge x_{i-|p[k]|+1}x_{i-|p[k]|+2} \dots x_i = p[k] & \text{altrimenti} \end{cases}$$

In base a questa rappresentazione formale, l'algoritmo può essere così descritto:

componibile(ITEM[] x , ITEM[][] p , integer n , integer m)	
boolean[] $T \leftarrow \mathbf{new\ integer}[0 \dots n]$	% Per ogni lunghezza i , riporta se è componibile o meno
integer[] $V \leftarrow \mathbf{new\ integer}[0 \dots n]$	% Per ogni lunghezza i , riporta la primitiva che fa da prefisso
$T[0] = \mathbf{true}$	
for $i \leftarrow 1$ to n do	
integer $k \leftarrow 1$	
$T[i] \leftarrow \mathbf{false}$	
while $k < m$ and $X[i] = \mathbf{false}$ do	
integer $j \leftarrow i - p[k] $	
if $x_{j+1}x_{j+2} \dots x_i = p[k]$ then	
$T[i] \leftarrow \mathbf{true}$	
$V[i] \leftarrow k$	
if $T[n]$ then	
printa (p, V, n)	
boolean stampa (ITEM[][] p , integer[] V , integer i)	
if $i = 0$ then return false	
if stampa ($p, V, i - p[V[i]] $) then	
print –	
print $P[V[i]]$	
return true	

Il vettore $V[i]$ contiene l'indice della stringa primitiva che forma il suffisso di $X(i)$. Questi valori vengono utilizzati al momento di stampare la composizione della stringa in stringhe primitive.

La complessità è $O(nl)$, dove n è la lunghezza di X ed l è la lunghezza totale delle stringhe primitive (nel caso pessimo, tutte le stringhe primitive dovranno essere esaminate fino all'ultimo carattere); si noti che $l = \Omega(m)$, in quanto assumiamo che le stringhe abbiano almeno un carattere.

Esercizio 2

È sufficiente ordinare gli intervalli per estremo di inizio, e poi controllare per ogni intervallo se c'è intersezione con quello successivo. Il costo è dato dall'ordinamento, $O(n \log n)$.

intersezione(integer[] l , integer[] u , integer n)	
{ ordina l, u per estremo di inizio }	
for $i \leftarrow 1$ to $n - 1$ do	
if $u[i] < l[i + 1]$ then	
return true	
return false	

Esercizio 3

Sviluppo l'iterazione e studio la sommatoria. Esplicito la costante “nascosta” in $\theta(\log n)$ scrivendo $c \log n$.

$$\begin{aligned}T(n) &= T(n-1) + c \log n \\&= T(n-2) + c \log(n-1) + c \log n \\&= T(n-3) + c \log(n-2) + c \log(n-2) + \log n \\&= T(1) + c \sum_{i=2}^n \log i\end{aligned}$$

La sommatoria può essere maggiorata nel modo seguente:

$$\begin{aligned}T(n) &= T(1) + c \sum_{i=2}^n \log i \\&\leq T(1) + c \sum_{i=1}^n \log n \\&= O(n \log n)\end{aligned}$$

La sommatoria può essere “minorata” nel modo seguente:

$$\begin{aligned}T(n) &= T(1) + c \sum_{i=2}^n \log i \\&= T(1) + c \sum_{i=1}^n \log i \\&\geq T(1) + c \sum_{i=1}^{n/2} \log 1 + c \sum_{i=n/2+1}^n \log n/2 \\&= T(1) + cn/2(\log n - 1) \\&= \Omega(n \log n)\end{aligned}$$

La funzione è quindi $\Theta(n \log n)$.

Esercizio 4

È possibile modificare semplicemente un algoritmo di visita in ampiezza per assegnare un livello ad ognuno dei nodi, e contare quanti nodi appartengono allo stesso livello. Il costo dell'algoritmo risultante è ovviamente $O(n)$.

```
visitaAmpiezza(TREE t)
integer larghezza  $\leftarrow$  1
integer level  $\leftarrow$  1
integer count  $\leftarrow$  1
QUEUE Q  $\leftarrow$  Queue()
Q.enqueue(t)
t.level  $\leftarrow$  0 while not Q.isEmpty() do
    TREE u  $\leftarrow$  Q.dequeue()
    if u.level  $\neq$  level then
        level  $\leftarrow$  t.level
        count  $\leftarrow$  0
    count  $\leftarrow$  count + 1
    if count > larghezza then larghezza  $\leftarrow$  count
    u  $\leftarrow$  u.leftmostChild()
    while u  $\neq$  nil do
        u.level  $\leftarrow$  t.level + 1 Q.enqueue(u)
        u  $\leftarrow$  u.rightSibling()
return larghezza
```

Cognome:

Nome:

Matricola:

Riga:

Col:

Algoritmi e Strutture Dati - Prova d'esame - Problemi

6/9/10

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna.

Esercizio 1 - Punti 6

Descrivere una struttura dati INSIEME che possa contenere interi compresi fra 1 ed n che supporti unicamente le operazioni INSERT(integer a) che inserisce il valore a nell'insieme e COUNT(integer a , integer b) che ritorna il numero di elementi contenuti nell'insieme e compresi nell'intervallo $[a, b]$ (estremi inclusi). Si assuma che $1 \leq a \leq b \leq n$. L'operazione COUNT() deve lavorare in tempo $O(1)$.

Esercizio 2 - Punti 8

Gli ordini di visita di un albero binario di 9 nodi sono i seguenti:

- A, E, B, F, G, C, D, I, H (anticipato)
- B, G, C, F, E, H, I, D, A (posticipato)
- B, E, G, F, C, A, D, H, I (simmetrico).

Si ricostruisca l'albero binario e si illustri *brevemente* il ragionamento.

Esercizio 3 - Punti 8

Si considerino le funzioni $f(n) = n$ e $g(n) = n^{1+\sin n}$. Si discuta se le due funzioni sono confrontabili in ordine di grandezza, ovvero:

1. $f(n) = O(g(n))$?
2. $f(n) = \Omega(g(n))$?
3. sia 1. che 2. sono vere ?
4. sia 1. che 2. sono false ?

Esercizio 4 - Punti 8+10

Bill Gates vi ha assunto per scrivere la nuova versione di Word, che verrà chiamata Word 2020. Siete responsabili per la procedura che gestisce la giustificazione del testo. Avete una sequenza di n parole di lunghezza $w_1 \dots w_n$ (misurata in punti tipografici), che dovete far entrare in righe di L punti. Non potete riordinare le parole. Se il testo è molto lungo, viene suddiviso in più righe. La somma K delle lunghezze delle parole contenute in una riga può essere inferiore o uguale ad L , ma *assolutamente* non più lunga. Una riga di lunghezza K ha penalità (positiva o nulla) $L - K$.

- **Parte (a)** Scrivete un algoritmo che divida il testo cercando di minimizzare la *penalità totale*, ovvero la sommatoria delle penalità delle righe così create.
- **Parte (b)** Scrivete un algoritmo che divida il testo cercando di minimizzare la *penalità massima*, ovvero la penalità più alta fra tutte le righe.

Dimostrare che i vostri algoritmi sono corretti. Calcolarne la complessità.

Una nota: l'algoritmo a) è effettivamente utilizzato in Word, l'algoritmo b) è utilizzato in LaTeX.

Un suggerimento: l'output può essere prodotto stampando le parole (**print** w_i) e stampando il carattere di new line (**println**).

Algoritmi e Strutture Dati - Prova d'esame

6/9/10

Esercizio 1

Una struttura dati di questo tipo può essere mantenuta tramite un array di interi $c[0 \dots n]$ di dimensione $n + 1$, in cui l'elemento i -esimo contiene il numero di elementi compresi fra 1 ed i , estremo inclusi. Contare gli elementi fra a e b si riduce quindi a ritornare $c[b] - c[a - 1]$. Aggiungere un elemento a richiede di verificare se $c[a] - c[a - 1]$ sia uguale a 0 (il che significa che a è assente), e in caso affermativo si aggiunge 1 a tutti gli elementi.

INSIEME

integer[] c

```
insieme insieme(integer  $n$ )
  INSIEME  $s \leftarrow$  new INSIEME
   $s.c \leftarrow$  new integer[0 ..  $n$ ]
  for  $i \leftarrow 0$  to  $n$  do
     $s.c[i] \leftarrow 0$ 
  return  $s$ 
```

```
insert(integer  $a$ )
  for  $i \leftarrow a$  to  $n$  do
     $c[i] \leftarrow c[i] + 1$ 
```

```
integer count(integer  $a$ , integer  $b$ )
  return  $c[b] - c[a - 1]$ 
```

Esercizio 2

È possibile ragionare nel modo seguente:

- A è la radice, perchè visitata per prima nell'ordine anticipato;
- A ha figlio sinistro e destro, perchè B,E,G,F,C appartengono al suo sottoalbero sinistro e D,H,I al suo sottoalbero destro (per la visita in ordine simmetrico);
- E è il figlio sinistro di A, perchè il primo ad essere visitato dopo A nella visita anticipata;
- B ha figlio sinistro e destro, perchè B appartiene al sottoalbero sinistro di E e G,F,C al suo sottoalbero destro (per la visita in ordine simmetrico);
- B è figlio sinistro di E (per la visita in ordine simmetrico);
- F è figlio destro di E (per la visita in ordine anticipato);
- G e C sono figli sinistri e destro di F (per la visita in ordine simmetrico);
- D è figlio destro di A (in quanto primo ad essere esaminato dopo il sottoalbero sinistro di A nell'ordine anticipato);
- Restano H e I; poichè I viene visitato prima di H nell'ordine anticipato, I deve essere padre di H e figlio di D;
- poichè I non viene visitato prima di D nell'ordine simmetrico, non può essere suo figlio sinistro; è quindi figlio destro di D;
- poichè H è visitato prima di I nell'ordine simmetrico, H deve essere figlio sinistro di I;

Esercizio 3

La funzione $\sin n$ oscilla fra -1 e $+1$; il che significa che la funzione “oscilla” fra il comportamento di n^{1-1} e n^{1+1} (si veda la figura). Per dimostrare che $f(n)$ è $O(g(n))$, dobbiamo trovare valori $n_0 > 0$, $c > 0$ tali che $f(n) \leq cg(n)$ per qualsiasi $n > n_0$. Ma $g(n)$ è periodicamente uguale a 1, quindi tali valori non possono esistere.

Per dimostrare che $f(n)$ è $\Omega(g(n))$, dobbiamo trovare valori $n_0 > 0$, $c > 0$ tali che $f(n) \geq cg(n)$ per qualsiasi $n > n_0$. Ma $g(n)$ è periodicamente uguale a n^2 , il che significa che tali valori non possono esistere.

La risposta corretta è la 4.

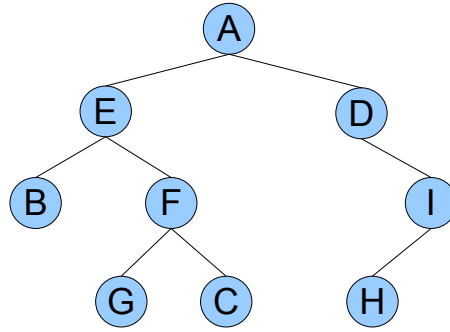


Figura 1: Albero risultante

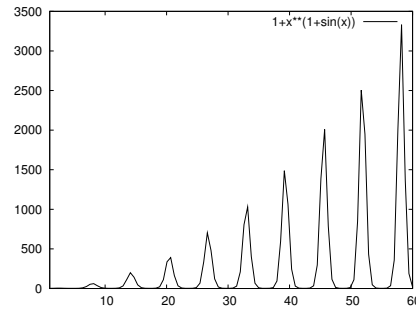


Figura 2: Funzione $n^{1+\sin n}$

Esercizio 4a

Descriviamo una soluzione come una collezione $P_1, \dots, P_m$ di insiemi di parole; P_i contiene l'insieme di parole che sono state stampate nella riga i -esima.

La penalità totale P_{tot} è data dalla seguente formula:

$$\begin{aligned}
 P_{tot} &= \sum_{1 \leq i \leq m} \left(L - \sum_{j \in P(i)} w_j \right) \\
 &= mL - \sum_{1 \leq i \leq m} \sum_{j \in P(i)} w_j \\
 &= mL - W
 \end{aligned}$$

In altre parole, la penalità totale dipende dal numero totale di righe, che deve essere minimizzato, e non dalla disposizione delle parole nelle stesse, in quanto la loro lunghezza è sempre W .

L'approccio utilizzato è quindi quello greedy; cerchiamo di riempire una riga parola per parola, fin quando c'è spazio residuo; dopo che si passa alla riga successiva. Per dimostrare che la scelta greedy è corretta, si consideri una soluzione ottima $P = P_1, P_2, \dots, P_m$ e supponiamo che la prima riga non sia riempita completamente. Si consideri una soluzione $P' = P'_1, P'_2, \dots, P'_m$ dove la prima riga è riempita completamente. La soluzione P' non può avere un numero inferiore di righe di P , altrimenti P non sarebbe ottima. Non ha sicuramente un numero di righe maggiore, perchè abbiamo semplicemente spostato alcune parole nella prima riga, dove vi era spazio. Quindi sia P che P' hanno la stessa penalità $mL - W$.

Il codice è il seguente (assumendo che tutte le parole siano più brevi della lunghezza L):

```
print(integer[] w, integer n, integer L)
integer t ← 0
integer i ← 1
while i ≤ n do
    while i ≤ n and t + w[i] ≤ L do
        print w_i
        t ← t + w[i]
        i ← i + 1
    println
```

Esercizio 4b

L'approccio greedy di cui al punto precedente non funziona se si vuole ridurre la penalità massima, invece che la penalità totale. Si consideri il seguente controesempio, con $L = 6$ caratteri:

aaa, bb
cc
dddd

In questo esempio, la penalità totale è $3 * 6 - (3 + 2 + 2 + 5) = 6$, mentre la penalità massima è $6 - 2 = 4$. Riordinando le parole in modo diverso, si ottiene il testo seguente con penalità massima $6 - 3 = 3$.

aaa
bb, cc
dddd

Per poter risolvere questo problema, è necessario utilizzare la programmazione dinamica. Definiamo la penalità di una riga composta dalle parole $w_i \dots w_j$ nel modo seguente:

$$c(i, j) = \begin{cases} L - \sum_{k=i}^j w_k & \text{se } \sum_{k=i}^j w_k \leq L \\ +\infty & \text{altrimenti.} \end{cases}$$

Definiamo una funzione $f(j)$ che riporta la minima penalità massima nella suddivisione in righe delle prime j parole:

$$f(j) = \begin{cases} c(1, j) & \text{se } c(1, j) < +\infty \\ \min_{1 \leq k < j} f(k) + c(k+1, j) & \text{altrimenti} \end{cases}$$

```
print(integer[] w, integer n, integer L)
integer[] f ← new integer[1..n]
integer[] p ← new integer[1..n]
for j ← 1 to n do
    if c(1, j) < +∞ then
        f[j] ← c(1, j)
        p[j] ← 0
    else
        integer pos ← 1
        for k ← 1 to j - 1 do
            if f[k] + c(k + 1, j) < f[pos] + c(pos + 1, j) then
                pos ← k
        f[j] ← f[pos] + c(pos + 1, j)
        p[j] ← pos
printRecursive(p, n)
```

```
printRecursive(integer[] p, integer i)
```

```
  if  $i \neq 0$  then  
    printRecursive( $p, p[i]$ )  
    for  $j \leftarrow p[i] + 1$  to  $i$  do print  $w_j$   
    println
```

Una variante più complessa dell'approccio con penalità massima è implementato in Latex, e produce risultati esteticamente migliori dell'approccio greedy implementato in Word.¹.

¹http://en.wikipedia.org/wiki/Word_wrap

Cognome:

Nome:

Matricola:

Riga:

Col:

Algoritmi e Strutture Dati - Prova d'esame - Problemi

9/1/11

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna.

Esercizio 1 - Punti 6

Sia dato un albero binario di ricerca T . Si assuma che a ciascun nodo v sia associato un colore $colore[v]$ tramite il vettore $colore$. Il colore può essere RED o BLACK. Scrivere un algoritmo che ritorna **true** se T è un albero Red-Black, **false** altrimenti.

Esercizio 2 - Punti 14

Bisogna costruire un aeroporto e si dispone di una mappa H della zona in cui va localizzato. La mappa suddivide il territorio in una griglia quadrata di $n \times n$ quadrati unitari, e per ogni quadrato unitario viene indicata l'altezza corrispondente. Bisogna localizzare la regione rettangolare di area massima (vale a dire la regione rettangolare che consiste nel maggior numero di quadrati) tale che il dislivello all'interno della regione non superi un dato limite C (i.e. la differenza d'altezza tra il quadrato di altezza massima della regione selezionata e quello d'altezza minima della regione selezionata deve essere al più C). Descrivere un algoritmo che presa la mappa H , e l'intero $C \ll n$ restituisce una regione richiesta (la regione da restituire può essere rappresentata tramite le coordinate della prima cella in alto a sinistra e le coordinate dell'ultima cella in basso a destra della regione stessa). Ad esempio per la mappa 8×8

```
20 18 21 21 20 18 23 20
19 17 20 21 16 18 20 21
-----
18 19|20 21 18 18 20 20|
21 17|18 18 21 21 19 19|
20 21|18 20 20 21 19 18|
18 20|19 18 21 20 18 21|
18 20|18 20 20 21 21 20|
-----
22 18 18 18 22 21 22 21
```

con $C = 4$ una soluzione è il rettangolo di lato 5 individuato dalle coordinate $(3, 3)$ e $(7, 8)$ ed evidenziato in figura.

Esercizio 3 - Punti 7

Sia data la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} k & \text{se } n \leq 2 \\ 4T(n/4) + \sqrt{n} & \text{se } n > 2 \end{cases}$$

Si fornisca una soluzione per $T(n)$ utilizzando il metodo di sostituzione.

Esercizio 4 - Punti 7

Siano X e Y due vettori di n interi, ordinati in senso non decrescente ed x un intero. Progettare un algoritmo che trovi, se esiste, una coppia di indici i, j tali che $X[i] + Y[j] = x$.

Algoritmi e Strutture Dati - Prova d'esame

9/1/11

Esercizio 1

Per verificare che un albero binario di ricerca sia effettivamente red-black, è necessario verificare le quattro proprietà che lo definiscono. La prima è semplice (radice nera), le altre richiedono delle sottoprocedure apposite.

```
boolean checkRedBlack(TREE T)
  if not color[T] = BLACK then return false
  if not check2(T) then return false
  if not check3(T) then return false
  if not check4(T) < 0 then return false
  return true
```

La proprietà 2 afferma che tutte le foglie devono essere nere. E' sufficiente visitare l'albero e verificare che i nodi che hanno entrambi puntatori sinistro e destro uguali a **nil** siano nere. Si noti che questo controllo non è necessario se si utilizza uno speciale nodo **NIL** per rappresentare le foglie.

```
boolean check2(TREE T)
  if T = nil then
    | return true
  if T.left = T.right = nil then
    | return color[T] = BLACK
  else
    | return check2(T.left) and check2(T.right)
```

La proprietà 3 afferma che i figli dei nodi rossi devono essere neri. E' facile verificarlo su tutti i nodi, in maniera ricorsiva.

```
boolean check3(TREE T)
  if T = nil or color[T]=BLACK then
    | return true
  else
    | return color[T.left] = color[T.right] = BLACK
```

La proprietà 4 afferma che la lunghezza nera di tutti i cammini radice-foglia deve essere uguale. Questa proprietà si applica ricorsivamente a tutti i cammini nodo-foglia, quindi è sufficiente valutare se la lunghezza nera dei cammini nodo-foglia dei figli destro e sinistro è uguale.

```
integer check4(TREE T)
  if T = nil then
    | return 0;
  integer hl ← check4(T.left)
  integer hr ← check4(T.right)
  if hl < 0 or hr < 0 or hl ≠ hr then
    | return -1;
  else
    | return iif(color[T] = RED, hl, hl + 1)
```

Esercizio 2

L'idea base è quella di calcolare, per ogni possibile coppia di coordinate (x_1, y_1) , (x_2, y_2) , lo scarto fra il valore minimo e il valore massimo presenti nella regione rettangolare compresa fra (x_1, y_1) e (x_2, y_2) (con $x_1 \leq x_2$ e $y_1 \leq y_2$).

L'approccio più ingenuo sarebbe quello di calcolare, per ogni possibile coppia di coordinate, il valore minimo e massimo ogni volta scorrendo tutte le celle fra esse comprese. Questo avrebbe una complessità di $O(n^6)$.

Si può invece sfruttare la programmazione dinamica per calcolare, in due tabelle min e max quadridimensionali, tali valori minimi e massimi, per poi calcolare lo scarto. Si usa in particolare memoization, e tutte le volte che si deve calcolare il minimo (massimo) di un rettangolo, si calcola il minimo fra il valore minimo (massimo) del rettangolo privato dell'ultima colonna e il valore minimo (massimo) dell'ultima colonna. Se il rettangolo è formato da una sola colonna, si calcola il valore minimo (massimo) fra il valore minimo (massimo) della colonna privata dell'ultimo elemento, e il valore dell'ultimo elemento. Se infine il rettangolo è formato da una singola cella, si ritorna quella.

Si noti che la funzione max è analoga alla funzione min rappresentata qui; si noti inoltre che assumiamo che le matrici min e max siano inizializzate a $\perp$. La complessità è $O(n^4)$, perchè ogni casella viene riempita al più una volta.

```

integer rettangolo(integer[][]  $H$ , integer  $n$ , integer  $C$ )
integer[][][]  $min \leftarrow$  new integer[1... $n$ ][1... $n$ ][1... $n$ ][1... $n$ ]
integer[][][]  $max \leftarrow$  new integer[1... $n$ ][1... $n$ ][1... $n$ ][1... $n$ ]
 $size \leftarrow 1$ 
for  $x_1 \leftarrow 1$  to  $n$  do
    for  $y_1 \leftarrow 1$  to  $n$  do
        for  $x_s \leftarrow x_1$  to  $n$  do
            for  $y_2 \leftarrow y_1$  to  $n$  do
                if  $\max(H, n, max, x_1, y_1, x_2, y_2) - \min(H, n, max, x_1, y_1, x_2, y_2) \leq C$  and  $(x_2 - x_1 + 1) \cdot (y_2 - y_1 + 1) > size$ 
                then
                     $size \leftarrow (x_2 - x_1 + 1) \cdot (y_2 - y_1 + 1)$ 
return  $size$ 

```

```

integer min(integer[][]  $H$ , integer[][][]  $min$ , integer  $n$ , integer  $x_1$ , integer  $y_1$ , integer  $x_2$ , integer  $y_2$ )
if  $min[x_1, x_2, y_1, y_2] = \perp$  then
    if  $x_1 = x_2$  and  $y_1 = y_2$  then
         $min[x_1, x_2, y_1, y_2] \leftarrow H[x_1][y_1]$ 
    else if  $x_1 = x_2$  then
         $min[x_1, x_2, y_1, y_2] \leftarrow \min(min[x_1, y_1, x_1, y_2 - 1], min[x_1, y_2, x_1, y_2])$ 
    else
         $min[x_1, x_2, y_1, y_2] \leftarrow \min(min[x_1, y_1, x_2 - 1, y_2], min[x_2, y_1, x_2, y_2])$ 
return  $min[x_1, x_2, y_1, y_2]$ 

```

E' possibile fare meglio di così? Probabilmente sì, ad esempio utilizzando un meccanismo divide-et-impera, dividendo la matrice in quattro parti $n/2 \times n/2$, cercando la più grande area contenuta in ognuna di esse e confrontandola con la più grande area contenuta in tutti i quattro quadranti.

Esercizio 3

Esercizio 4

Vi sono ovviamente varie soluzioni.

(1) Una soluzione banale effettua la somma di tutte le coppie di numeri, verificando se il valore risultante è uguale a x , con un costo $O(nm)$ se i vettori hanno dimensione n, m .

(2) Una versione migliore prende ogni elemento del vettore X , e cerca il valore $x - X[i]$ nel vettore Y tramite ricerca binaria. Il costo di questo algoritmo è $O(n \log m)$, oppure $O(\min\{n, m\} \log \max\{n, m\})$, assumendo di fare la scelta (più efficiente) di scorrere tutti gli elementi del vettore più piccolo e cercare nel vettore più grande.

(3) Una versione ancora più efficiente, con costo $O(n + m)$, è la seguente:

boolean search(integer[] X , integer[] Y , integer n , integer m , integer x)

```
integer  $i \leftarrow 1$ 
integer  $j \leftarrow m$ 
while  $i \leq n$  and  $j \geq 1$  do
    if  $X[i] + Y[j] = x$  then
         $\perp$  return true
    else if  $X[i] + Y[j] > x$  then
         $\perp$   $j \leftarrow j - 1$ 
    else
         $\perp$   $i \leftarrow i + 1$ 
return false
```

In questa versione, l'idea è la seguente: siano $X[1 \dots n]$ e $Y[1 \dots m]$ i vettori considerati. Si prende la somma $X[1] + Y[m]$; se questa è pari a x , abbiamo fatto e possiamo restituire **true**. Altrimenti, ci sono due casi: se la somma è maggiore di x , $Y[m]$ non può essere associato con alcun elemento di X , perchè $X[1]$ è il minimo di X . Possiamo quindi considerare il sottoproblema $X[1 \dots n], Y[1 \dots m-1]$. Se invece la somma è minore di x , $X[1]$ non può essere associato ad alcun elemento elemento di Y , perchè $Y[m]$ è il massimo di Y . Possiamo quindi considerare il sottoproblema $X[2 \dots n], Y[1 \dots m]$.

Algoritmi e Strutture Dati - Prova d'esame - Problemi
31/1/11

Esercizio 0 Scrivere correttamente nome, cognome, numero di matricola, riga e colonna.

Esercizio 1 - Punti 7

Sia $G = (V, E)$ un grafo non orientato, connesso e pesato. Si supponga di aver determinato un minimo albero di copertura di G , T , avente radice t . Si assuma che il peso dell'arco $(u, v) \in E$ venga decrementato di 1 e si consideri il problema di determinare, dato T , un minimum spanning tree del grafo modificato. Si definisca un procedimento efficiente per risolvere il problema proposto e se ne valutino correttezza e complessità.

Esercizio 2 - Punti 14

Sia V un vettore di interi. Si considerino sottovettori contigui (senza interruzioni) e non nulli (di lunghezza almeno 1) di V . Chiamiamo costo di un sottovettore di V la somma dei suoi elementi. Data una partizione in sottovettori del vettore V , chiamiamo costo della partizione il costo massimo dei suoi sottovettori. Dato un vettore V di n interi e un intero k con $2 \leq k \leq n$, il problema è trovare una partizione di V in esattamente k sottovettori, il cui costo sia minimo.

- (a) Descrivere un algoritmo che risolve il problema nel caso particolare $k = 2$ e se ne valutino correttezza e complessità.
- (b) Descrivere un algoritmo che risolve il problema nel caso particolare $k = 3$ e se ne valutino correttezza e complessità.

Esercizio 3 - Punti 7

Sia data la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} k & \text{se } n \leq 2 \\ 2T(n/4) + \log n & \text{se } n > 2 \end{cases}$$

Si fornisca una soluzione per $T(n)$ senza utilizzare il Master Theorem (o Teorema dell'Esperto).

Esercizio 4 - Punti 7

Sia A un vettore di n interi e si consideri il problema di determinare se esistono due interi che occorrono in A lo stesso numero di volte. Si scriva un algoritmo che risolve il problema e se ne valutino correttezza e complessità.

Algoritmi e Strutture Dati - Prova d'esame

31/1/11

Esercizio 1

Posso darsi due casi. Se l'arco (u, v) che viene decrementato fa già parte del minimo albero di copertura, quest'ultimo è un minimo albero di copertura anche per il nuovo grafo. Infatti, quando (u, v) è stato scelto, era l'arco con peso minimo fra tutti quelli che potevano essere aggiunti; siccome il nuovo peso è inferiore a quello precedente, il peso resta minimo.

Se invece (u, v) non appartiene al minimo albero di copertura, lo aggiungiamo all'albero ottenendo un ciclo. Si cerca quindi l'arco di peso minimo fra quelli appartenenti al ciclo e si ottiene un nuovo albero di copertura (potenzialmente uguale al primo).

Per individuare un ciclo, è sufficiente utilizzare una visita del grafo ottenuto aggiungendo (u, v) all'arco di copertura, con costo $O(m+n)$. Per individuare l'arco con peso minimo nel ciclo, è sufficiente scorrere tutti gli archi che vi appartengono con costo $O(n)$. Il costo totale è quindi $O(m+n)$.

Esercizio 2

Parte (a), $k = 2$ L'idea è la seguente: innanzitutto calcoliamo il costo dell'intero vettore A . Dopo di che, calcoliamo tutte le possibile partizioni in due sottovettori $A[1 \dots i]$ e $A[i+1 \dots n]$, $1 \leq i \leq n-1$, calcolando il costo di $A[1 \dots i]$ aggiornandolo ad ogni incremento di i , e calcolando il costo di $A[i+1 \dots n]$ come differenza fra il costo totale e il costo di $A[1 \dots i]$. Il costo di questo algoritmo è ovviamente $O(n)$.

integer minCost2(**integer**[] A , **integer** n)

```
integer tot  $\leftarrow$  0
for  $i = 1$  to  $n$  do  $tot \leftarrow tot + A[i]$ 
integer minPos  $\leftarrow$  0
integer minCost  $\leftarrow$   $+\infty$ 
integer par  $\leftarrow$  0
for  $i = 1$  to  $n - 1$  do
     $par \leftarrow par + A[i]$ 
    if  $\max(par, tot - par) < minCost$  then
         $minPos \leftarrow i$ 
         $minCost \leftarrow \max(par, tot - par)$ 
return minPos
```

Parte (b), $k = 3$ L'algoritmo è simile nell'idea, ma questa volta le posizioni in cui interrompere il vettore sono due, identificate dagli indici i e j ; i sottovettori sono quindi $A[1 \dots i]$, $A[i+1 \dots j]$, $A[j+1 \dots n]$, con $1 \dots i \dots n-2$ e $i \dots j \dots n-1$. Poichè in questa procedura abbiamo due indici annidati, il costo della procedura è $O(n^2)$.

integer minCost3(**integer**[] A , **integer** n)

```
integer tot  $\leftarrow$  0
for  $i = 1$  to  $n$  do  $tot \leftarrow tot + A[i]$ 
integer minPos1  $\leftarrow$  minPos2  $\leftarrow$  0
integer minCost  $\leftarrow$   $+\infty$ 
integer par1  $\leftarrow$  0
for  $i = 1$  to  $n - 2$  do
     $par1 \leftarrow par1 + A[i]$ 
    integer par2  $\leftarrow$  0
    for  $j = i + 1$  to  $n - 1$  do
         $par2 \leftarrow par2 + A[j]$ 
        if  $\max(par1, par2, tot - par1 - par2) < minCost$  then
             $minPos1 \leftarrow i$ 
             $minPos2 \leftarrow j$ 
             $minCost \leftarrow \max(par1, par2, tot - par1 - par2) \leq minCost$ 
return  $\langle minPos1, minPos2 \rangle$ 
```

Esercizio 3

Esercizio 4

Un algoritmo semplice ordina gli n numeri, con un costo pari a $O(n \log n)$. Dopo di che, crea un vettore di supporto B di dimensione n (perché al limite vi sono n numeri distinti nel vettore) e scorre il vettore A ordinato contando quante volte (consecutive) si ripete un valore e scrivendo tale numero nel vettore B . Dopo di che, ordina B (costo $O(n \log n)$) e cerca se esistono valori ripetuti, nel qual caso risponde **true**.

boolean sameNumber(**integer**[] A , **integer** n_A)

```
sort( $A, n_A$ )
integer[]  $B \leftarrow$  new integer[1 . . .  $n_A$ ]
integer  $count \leftarrow 1$ 
integer  $n_B \leftarrow 1$ 
for  $i = 2$  to  $n_A$  do
    if  $A[i - 1] \neq A[i]$  then
         $B[n_B] \leftarrow count$ 
         $count \leftarrow 1$ 
         $n_B \leftarrow n_B + 1$ 
 $B[n_B] = count$ 
sort( $B, n_B$ )
for  $i = 2$  to  $n_B$  do
    if  $B[i - 1] = B[i]$  then
        return true
return false
```

1 Problemi

1.1 Differenze minime e massime

Sia V un vettore di n numeri interi positivi. Definiamo i seguenti due problemi computazionali:

1. **min-gap** (differenza minima): determinare una coppia di indici $1 \leq i < j \leq n$ tali per cui per ogni $1 \leq k < h \leq n$: $|V[i] - V[j]| \leq |V[k] - V[h]|$.
2. **max-gap** (differenza massima): determinare una coppia di indici $1 \leq i < j \leq n$ tali per cui per ogni $1 \leq k < h \leq n$: $|V[i] - V[j]| \geq |V[k] - V[h]|$.

Risolvere i due problemi e discutere la complessità delle soluzioni.

1.2 Per fare un albero (binario di ricerca) ci vuole...

Dato un vettore V di n interi ordinati e distinti, scrivere una procedura che costruisce un albero binario di ricerca di altezza minima. Discuterne la correttezza e la complessità.

1.3 Ricorrenza

Trovare un limite superiore e inferiore per la seguente ricorrenza:

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + 1 & \text{se } n > 1 \text{ è pari} \\ T(n-2) + 1 & \text{se } n > 1 \text{ è dispari} \end{cases}$$

Suggerimento: utilizzate pure i teoremi per avere un'idea della soluzione, ma poi sarà necessario utilizzare il metodo di sostituzione per una dimostrazione formale.

1 Soluzioni

1.1 Differenze minime e massime

min-gap : Il problema ha ovviamente una soluzione di complessità $O(n^2)$, che confronta tutte le possibile coppie di indici e i rispettivi valori. E' possibile fare meglio?

E' possibile notare che ordinando i valori, è possibile scorrere il vettore confrontando posizioni consecutive, alla ricerca della coppia di valori più vicina. Così facendo però si perde l'informazione sulla posizione dei valori nel vettore originale. In ogni caso, l'ordinamento costa $O(n \log n)$; una volta individuati i due valori più vicini, è possibile cercarli in tempo $O(n)$ nel vettore originale.

```
(integer, integer) min-gap(integer[] A, integer n)
integer[] B ← newinteger[1..n]
for i ← 1 to n do B[i] ← A[i]
quicksort(B, n)

integer min ← 2
for i ← 3 to n do
    if B[i] - B[i - 1] < B[min] - B[min - 1] then
        min ← i

integer i1 ← 0
integer i2 ← 0
for i ← 1 to n do
    if i1 = 0 and A[i] = A[min - 1] then
        i1 ← i
    else if i2 = 0 and A[i] = A[min] then
        i2 ← i
return (i1, i2)
```

Altrimenti, si potrebbe modificare un algoritmo di ordinamento in modo da mantenere, parallelamente ai valori, la loro posizione all'interno del vettore originale. Una volta ordinati i valori, è sufficiente scorrerli tutti cercando la coppia di elementi consecutivi con differenza minima.

La complessità dell'algoritmo risultante è dominata dall'ordinamento, che ha costo $O(n \log n)$.

max-gap : Il problema è più semplice; si tratta di individuare l'indice in cui si trova l'elemento minimo e l'indice in cui si trova l'elemento massimo.

```
(integer, integer) max-gap(integer[] A, integer n)
integer i1 ← min(V, n); % Restituisce indice del minimo
integer i2 ← max(V, n); % Restituisce indice del massimo
return (i1, i2)
```

Ovviamente un algoritmo del genere ha costo lineare nel numero di elementi $O(n)$.

1.2 Per fare un albero (binario di ricerca) ci vuole...

L'idea è prendere il valore nella posizione mediana e inserirlo nell'albero; questo fa sì che i due sottoalberi sinistro e destro abbiano dimensioni simili (differiscono al più di 1) e quindi se costruiti in maniera bilanciata, fanno sì che l'albero risultante sia bilanciato.

(integer, integer) build-tree-rec(TREE T , integer[] A , integer i , integer j)

```
if  $i < j$  then
    integer  $m \leftarrow (i + j)/2$ 
    TREE  $u \leftarrow T.\text{insertNode}(V[m], V[m])$ 
    build-tree-rec( $u, A, i, m - 1$ )
    build-tree-rec( $u, A, m + 1, j$ )
```

(integer, integer) build-tree-rec(integer[] A , integer n)

```
TREE  $T \leftarrow \text{Tree}()$ 
build-tree-rec( $T, A, 1, n$ )
```

Alcune cose da notare:

- Nel libro, insertNode() ritorna il puntatore alla radice dell'albero. In questo caso, il costo computazionale dell'algoritmo è $O(n \log n)$, perchè ogni inserimento deve trovare la posizione corretta a partire dalla radice (costo $O(\log n)$)
- Se invece si assume che insertNode() ritorni il nuovo nodo appena creato, la complessità dell'algoritmo è $O(n)$, perchè ogni inserimento deve semplicemente scegliere se mettere un valore a destra o a sinistra. Se così è, bisogna specificarlo nel compito.
- Una possibilità alternativa è scrivere codice ad-hoc per la gestione dell'albero per questo problema. La mia versione tende a minimizzare il codice scritto.
- Notate che rispetto alla versione che ho presentato a lezione, ho aggiunto l'ipotesi che i numeri siano distinti. Questo semplifica la gestione degli ABR.

1.3 Ricorrenza

Bisogna affrontare i due casi (pari e dispari) separatamente; utilizzando i teoremi visti a lezione, è facile vedere che $T(n) = \Theta(\log n)$ nel caso delle potenze di 2, mentre $T(n) = \Theta(n)$ nel caso dei numeri dispari (infatti, nel caso delle potenze di 2 si applica solo la divisione per 2, che produce ancora una potenza di 2, mentre nel caso dei numeri dispari si applica solo la sottrazione di 2, che produce ancora un numero dispari). Cercando un limite inferiore e superiore, è facile quindi capire che un limite inferiore per la ricorrenza è pari a $\Omega(\log n)$, mentre un limite superiore è $O(n)$.

Dimostriamo per sostituzione il limiter superiore: vogliamo dimostrare che $\exists c > 0, \exists m \geq 0$ tale che $T(n) \leq cn$ per ogni $n \geq m$.

Se n è pari,

$$\begin{aligned} T(n) &= T(n/2) + 1 \\ &\leq cn/2 + 1 \\ &\leq cn \end{aligned}$$

che è vera per $c \geq 2/n$, che è soddisfacibile per $c = 1$ con $m = 2$.

Se n è dispari,

$$\begin{aligned}T(n) &= T(n-2) + 1 \\&\leq c(n-2) + 1 \\&\leq cn\end{aligned}$$

che è vera per $2c \geq 1$, ovvero per $c \geq 1/2$ per tutti gli n .

Il caso base è vero per $T(n) = 1 \geq 1 \cdot c$, ovvero per $c \geq 1$. Quindi è banalmente dimostrato che per $c = 1$, $m = 2$, $T(n) \leq cn$ per ogni $n \geq m$.

La dimostrazione per il limite inferiore è analoga: dobbiamo dimostrare che $\exists c > 0, \exists m \geq 0$ tale che $T(n) \geq c \log n$ per ogni $n \geq m$.

Se n è pari,

$$\begin{aligned}T(n) &= T(n/2) + 1 \\&\geq c \log(n/2) + 1 && \text{per sostituzione} \\&= c \log n - c + 1 \\&\geq c \log n\end{aligned}$$

che è vera per tutti i $c \leq 1$ e per tutti gli n .

Se n è dispari,

$$\begin{aligned}T(n) &= T(n-2) + 1 \\&\geq c \log(n-2) + 1 && \text{per sostituzione} \\&\geq c \log(n/2) + 1 && \text{per } m \geq 4 \\&\geq c \log n\end{aligned}$$

che è vera per tutti i $c \leq 1$ e per tutti gli $n \geq m = 4$.

Il caso base dà origine a problemi, ma è sufficiente utilizzare $n = 2, 3$ come casi base e vedere che

$$\begin{aligned}T(2) = T(1) + 1 &= 2 \geq 2c \\T(3) = T(1) + 1 &= 2 \geq 3c\end{aligned}$$

che sono entrambe soddisfatte per $c \geq 1$.

Quindi, per $c = 1$, $m = 4$, il nostro limite inferiore è provato.